



Basys3 Neural Network Accelerator

Hayden Thai
823877568

Electrical/Computer Engineering

May 1st, 2025

TABLE OF CONTENTS

1. Introduction.....	4
2. High-Level Design.....	5
2.1 Overall System Block Diagram	5
2.2 Internal FPGA Block Diagram	6
3. Functional Description.....	6
3.1 PicoRV32 CPU (picorv32_wrapper)	6
3.2 Neural Network Accelerator (nn_accelerator).....	6
3.3 SRAM Controller (sram_controller).....	7
3.4 Flash Controller (flash_controller)	7
3.5 MMIO Module (mmio).....	7
3.6 Seven Segment Display Driver (seven_seg).....	8
3.7 UART Interface (uart_rx, uart_tx, top_uart).....	8
3.8 SoC Top Level (soc_top).....	8
4. Memory Initialization	8
4.1 C Program Instruction Generation	8
5.2 Image and Weight Generation	11
5. Toolchain Used	15
6. Memory Map	15
6.1 Overall 32-bit Address Space Map (Von Neumann Architecture).....	15
6.2 SRAM Layout (0x0000_0000 – 0x000F_FFFF)	15
6.3 MMIO Address Map (0x4000_0000 – 0x4000_0FFF)	16
6.4 Flash Memory Layout (0x8000_0000 – 0x800F_FFFF).....	16
7. Test Bench and Validation.....	16
7.1 Test Bench Description.....	16
7.2 Simulation Results	17
7.2.1 Program Initialization	17
7.2.2 Image and Weight Copy from Flash to Sram	17
7.2.3 Neural Network Inferencer Starts	19
7.2.4 Neural Network Module Starts Reading Image Data	19
7.2.5 Layer 1 Weight Access	20
7.2.6 Layer 2 Weight Access	20
7.2.7 Layer 3 Weight Access	21
7.2.8 Argmax Phase (after Layer 3 Completion).....	21
7.3 Project Limitations and Bugs.....	22
7.4 Worst Case Analysis	23
8. Project Development Log	23
9. Special Sources or IP Used.....	23
9.1 PicoRV32.....	23
10. Improvements	23
11. Conclusion	24
12. References.....	24
13. Source Code.....	25
13.1 soc_top.v	25

13.2 picorv32_wrapper.v	28
13.3 flash_controller.v	29
13.3 mmio.v	31
13.4 sram.v.....	32
13.5 nn_accelerator.v	35
13.6 seven_seg.v	42
13.7 uart_top.v	42
13.8 tb_soc.v	48
13.9 main.c.....	48
13.10 start.S	50
13.11 linker.ld	50
13.12 re-assembled assembly.txt	51
13.13 weights_and_image.py.....	53

LIST OF FIGURES

Figure 1: Overall High Level FPGA Design	5
Figure 2: Internal FPGA Implementation	6
Figure 3: Grayscale Test Image	12
Figure 4: Top Level Testbench Overview	16
Figure 5: CPU Program Boot.....	17
Figure 6: CPU starts to write image data from Flash to SRAM.....	18
Figure 7: CPU starts to write 1 st layer weights from flash to SRAM	18
Figure 8: CPU starts to write 2 nd layer weights from flash to SRAM.....	18
Figure 9: CPU starts to read 3rd layer weights from flash to SRAM.....	19
Figure 10: mmio_start is triggered and neural network inferencing starts	19
Figure 11: Neural Network Starts Inferencing Image Data	20
Figure 12: First Non-Zero Image Data	20
Figure 13: Neural Network Starts Inferencing Layer 1 Weights.....	20
Figure 14: Neural Network Starts Inferencing Layer 2 Weights.....	21
Figure 15: Neural Network Starts Inferencing Layer 3 Weights.....	21
Figure 16: Neural Network Final Phase(Argmax).....	21

1. Introduction

In the field of computing, the acceleration of machine learning tasks has become an active area of research. With the increasing computational demands of AI, solutions that provide high efficiency, low latency, and low power consumption are being explored. This project focuses on the development of a neural network inferencer implemented entirely on an FPGA platform using the Basys3.

The project involved designing and implementing an SRAM controller, Flash controller, a soft-core processor based on the PicoRV32 RISC-V CPU, and a neural network accelerator. The system is capable of loading image and weight data from flash memory into SRAM, inferencing a neural network, and outputting its classification result. The neural network architecture consists of three fully connected layers, each performing multiply-accumulate operations followed by ReLU activations, with the final layer producing a prediction.

FPGAs are ideal for applications requiring custom data paths and parallel processing. Unlike general-purpose processors, an FPGA allows for highly efficient, specialized hardware tailored to specific computational tasks such as neural network inference. This enables significant improvements in latency, power consumption, and throughput.

2. High-Level Design

2.1 Overall System Block Diagram

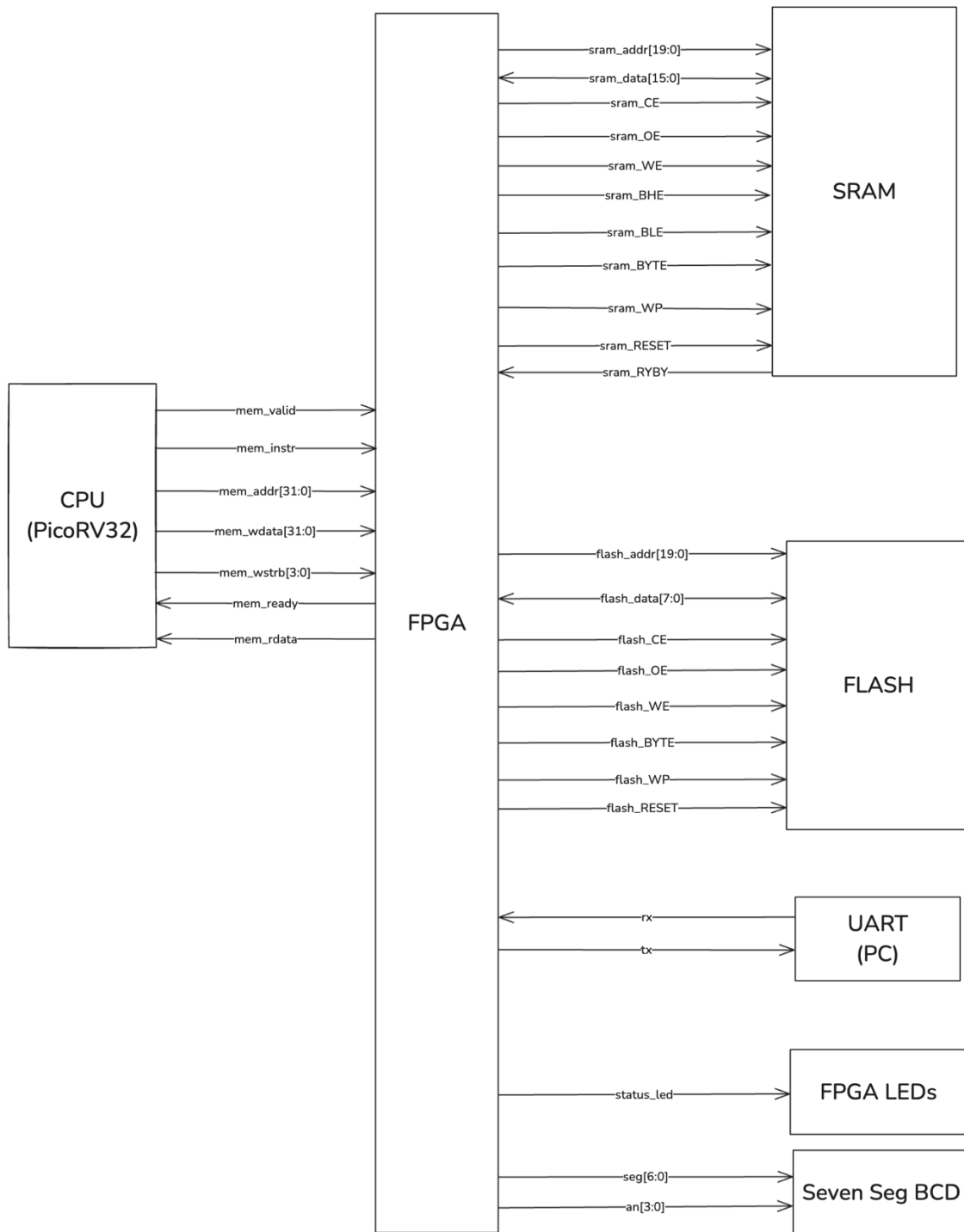


Figure 1: Overall High Level FPGA Design

2.2 Internal FPGA Block Diagram

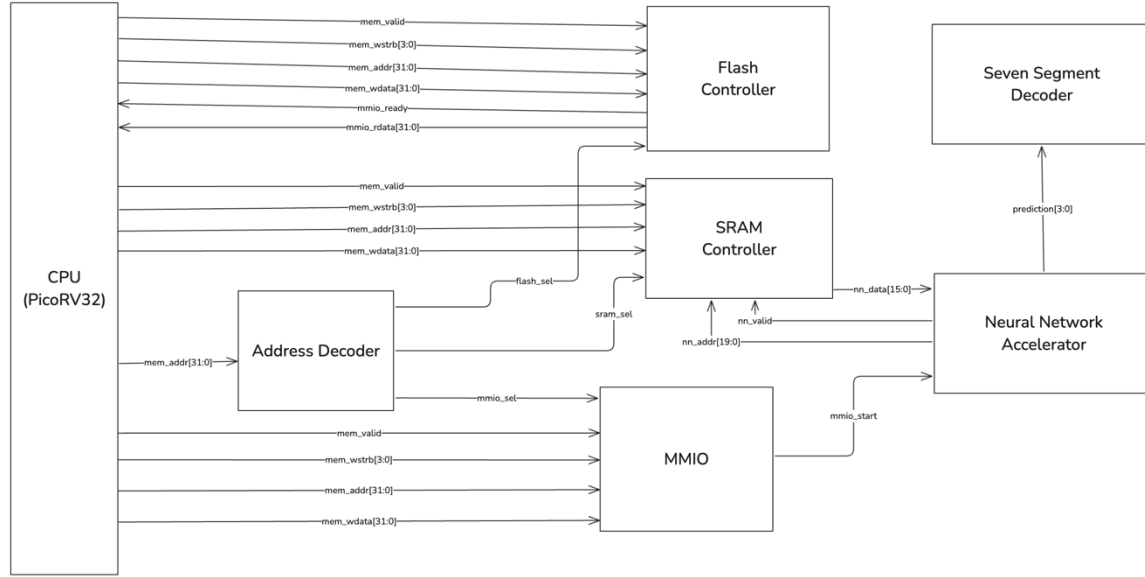


Figure 2: Internal FPGA Implementation

3. Functional Description

3.1 PicoRV32 CPU (picorv32_wrapper)

The PicoRV32 CPU is a small processor core based on the RISC-V instruction set, responsible for executing the software that coordinates system behavior. In this project, it is instantiated within a wrapper module (picorv32_wrapper) that connects its memory interface to the rest of the SoC. The CPU executes firmware from flash memory, which includes copying image data and neural network weights into SRAM. It then starts the neural network accelerator by writing to a control register via **memory mapped I/O** (MMIO). The processor fetches instructions, performs data movement, and triggers system functions through memory-mapped operations.

3.2 Neural Network Accelerator (nn_accelerator)

The nn_accelerator module infers a fixed three-layer fully connected neural network. It processes a 28x28 image stored in SRAM and produces a predicted number(0-9) as output. The module uses SRAM through a read-only port and is initiated by an external start signal triggered by mmio. Once triggered, it sets a busy flag and begins fetching the input image from 0x10000(mmio specified image_base address). The image is internally buffered before processing.

The accelerator computes layer outputs sequentially. Each layer consists of a series of multiply-accumulate (MAC) operations over its input vector and a corresponding weight row, which includes one bias term followed by weights. Biases and weights are read from SRAM using base addresses provided for each layer. All multiply-accumulate operations are performed using 16-bit signed integers and a 32-bit Q16.16 fixed-point accumulator to preserve accuracy.

After accumulating the weighted sum, each neuron applies a ReLU activation (zeroing negative values) in the first two layers. The final layer produces raw output scores without activation. The accelerator then performs an argmax operation across the 10 output logits to determine the predicted class, which is output through a 4-bit prediction register.

Control flow is managed by a finite state machine (FSM) that advances through all image loading, MAC processing, wait cycles for SRAM reads, activation, and argmax calculation. Internal arrays store intermediate results between layers, and a configurable number of wait cycles ensures correct timing for memory fetches. When inference completes, the busy signal is cleared, and the result can be read by external modules or used to drive a display.

3.3 SRAM Controller (*sram_controller*)

The `sram_controller` module arbitrates access to a shared single-port SRAM between the CPU and the neural network accelerator. It supports 32-bit read and write transactions from the CPU and 16-bit read-only transactions from the accelerator. In simulation mode, it internally models a 1MB 16-bit SRAM array using a behavioral memory array. CPU accesses require address translation and data packing or unpacking: 32-bit words are split across two 16-bit memory locations, and byte-order is adjusted for little endian. The accelerator performs 16-bit reads directly using byte-addressed addresses, with indexing handled internally by shifting addresses. The controller dynamically selects between CPU and NN requests based on the `nn_valid` signal. When the accelerator is active, its address is used. Otherwise, the CPU is serviced.

3.4 Flash Controller (*flash_controller*)

The flash controller provides access between the MMIO bus and an emulated flash memory. During simulation, it models a 1MB flash device using an internal byte-addressed array, initialized with contents from `flash_init.mem`. When a valid MMIO request is received, the module asserts `mmio_ready` and services the transaction. For reads, it computes a byte offset by subtracting the flash base address (0x80000000) from the MMIO address, aligns it to a 32-bit boundary, and returns a 32-bit word by combining four adjacent bytes in little-endian order. Write operations are ignored, but the input data is echoed back on `mmio_rdata`.

3.5 MMIO Module (*mmio*)

The `mmio` module is a minimal memory mapped I/O interface used by the CPU to control the neural network accelerator and peripheral signals. It responds to transactions targeting the address range beginning at 0x40000000. When a write to offset 0x00 is detected with a value of 1, the module asserts the `mmio_start` signal for one cycle to trigger the accelerator and the written value is echoed back on `mmio_rdata`. For all other reads, it returns zero. The `mmio_ready` signal is asserted immediately for all valid transactions.

3.6 Seven Segment Display Driver (*seven_seg*)

The seven-segment module takes a 4-bit digit input and outputs a 7-bit seg signal to control a common-anode 7-segment display and a 4-bit an signal for digit enable control. It uses a combinational always block to decode the digit value into the corresponding 7-segment pattern, with seg_reg defining which segments are active (active low). The an_reg output is hardcoded to 4'b1110, activating only the rightmost digit by setting its enable line low while keeping the others high.

3.7 UART Interface (*uart_rx*, *uart_tx*, *top_uart*)

The UART modules were reused from a previous COMPE470L lab assignment and were originally intended to implement a serial loopback and inference control system. The plan was to receive image data over rx_serial, store it in SRAM, trigger the neural network accelerator, and transmit the prediction result back via tx_serial. The module supports 9600 baud with a 100 MHz clock, and includes Rx/Tx modules with baud-aligned timing.

3.8 SoC Top Level (*soc_top*)

The top-level module connects the PicoRV32 CPU, flash controller, sram controller, mmio interface, neural network accelerator, and seven-segment display driver into a complete system. It accepts clock and reset inputs and drives output peripherals including a seven-segment display and a status LED. CPU memory transactions are routed based on the upper address nibble: addresses starting with 0x8 access the flash controller (for simulation, an internal flash model; in synthesis, the interface is unconnected), 0x4 access the MMIO controller, and all others access the SRAM controller. Read and ready signals from all memory domains are multiplexed back to the CPU. The SRAM controller arbitrates access to a shared single-port SRAM, giving priority to the neural network accelerator when active. The MMIO module handles control transactions, writing 1 to offset 0x00 triggering a start signal for the accelerator. The accelerator reads image and weight data from SRAM, performs inference using three fully connected layers, and outputs a 4-bit class prediction (which corresponds to numbers 0-9). This result is displayed via the seven_seg module. The status_led is driven directly by the accelerator's busy flag. UART signals are present in the top-level port list but remain unconnected in this system configuration.

4. Memory Initialization

4.1 C Program Instruction Generation

The memory initialization process starts with the CPU booting from flash memory mapped at 0x80000000, defined in linker.ld under the FLASH memory region:

```
MEMORY
{
    FLASH (rx) : ORIGIN = 0x80000000, LENGTH = 256K /* Flash for firmware */
```

```

    SRAM (rwx) : ORIGIN = 0x00010000, LENGTH = 128K    /* SRAM for data, image,
weights, stack */
}

```

Upon reset, the CPU begins execution at `_start` (0x80000000), where `start.S` initializes the stack pointer using `_stack_top`, then jumps to `main()`.

```

.section .init
.globl _start
_start:
    la      sp, _stack_top    # Load stack pointer from _stack_top in the linker
script
    call    main              # Call main()
1: j       1b                 # Infinite loop if main returns

```

The firmware (`main.c`) calculates `FIRMWARE_SIZE` by subtracting `FLASH_BASE` from the linker-generated `_etext` symbol.

```

// The firmware code is linked at FLASH_BASE.
// _etext is defined by the linker as the end of the firmware code
extern char _etext[];
#define FIRMWARE_SIZE      ((uint32_t)_etext - FLASH_BASE)

```

It then copies the MNIST image and three weight matrices from flash to SRAM at predefined addresses. When reading from flash, the memory-mapped flash controller assembles 32-bit words from four consecutive bytes in little-endian order. This is handled by the flash controller, which interprets addresses offset from 0x80000000 and returns combined 8-bit data using `$readmemh("flash_init.mem")` for initialization

```

// SRAM addresses where the data will be copied
#define IMAGE_BASE          0x00010000    // MNIST image data
#define WEIGHT1_BASE       0x00020000    // Layer 1 weights
#define WEIGHT2_BASE       0x00030000    // Layer 2 weights
#define WEIGHT3_BASE       0x00040000    // Layer 3 weights

// Copy image data (flash is laid out with firmware first, then image data)
copy_memory(FLASH_BASE + FIRMWARE_SIZE, IMAGE_BASE, IMAGE_WORDS);

// Copy weights data to SRAM.
// Each 32-bit word is 4 bytes, so we multiply word counts by 4 when computing byte
offsets
copy_memory(FLASH_BASE + FIRMWARE_SIZE + IMAGE_WORDS * 4, WEIGHT1_BASE,
WEIGHT1_WORDS);
copy_memory(FLASH_BASE + FIRMWARE_SIZE + (IMAGE_WORDS + WEIGHT1_WORDS) * 4,
WEIGHT2_BASE, WEIGHT2_WORDS);

```

```
copy_memory(FLASH_BASE + FIRMWARE_SIZE + (IMAGE_WORDS + WEIGHT1_WORDS +
WEIGHT2_WORDS) * 4, WEIGHT3_BASE, WEIGHT3_WORDS);
```

Each transfer copies 32-bit words, where the word counts have been preadjusted to account for the accelerator's 16-bit fixed-point data. After initialization, the CPU writes 1 to the accelerator control register at 0x40000000, then busy-waits for the computation to complete:

```
// Start the accelerator by writing '1' to its control register.
ACCEL_CTRL = 1;

// Wait until the accelerator clears the control register (indicating nn finished
inferencing)
while (ACCEL_CTRL != 0) {}
```

The prediction result is read from ACCEL_OUT and written to the seven-segment display register SEVEN_SEG:

```
uint32_t prediction = ACCEL_OUT;
SEVEN_SEG = prediction;
```

After a while I automated the build process in a Makefile, where start.S and main.c are compiled into firmware.elf using riscv64-unknown-elf-gcc, linking against linker.ld:

```
CC = riscv64-unknown-elf-gcc
CFLAGS = -Os -march=rv32im -mabi=ilp32 -nostartfiles -T linker.ld -mno-relax

firmware.elf: start.S main.c linker.ld
    $(CC) $(CFLAGS) -o firmware.elf start.S main.c
```

The CFLAGS options do the following:

- -Os: Optimize for size, reducing firmware footprint in flash.
- -march=rv32im: Targets a RISC-V RV32I core with multiplication (M) extension enabled.
- -mabi=ilp32: Specify the integer ABI where integers, longs, and pointers are all 32 bits.
- -nostartfiles: Omit the standard system startup files so that the linker uses start.S as the sole entry point.
- -T linker.ld: Use the linker script to control memory layout.

- `-mno-relax`: Disable linker relaxation optimizations, ensuring consistent instruction sequences between compile time and link time, critical for bare-metal embedded environments.

After compilation, `firmware.elf` is stripped into a raw binary (`firmware.bin`). This binary is then concatenated with externally generated image and weight binaries (`image.bin` and `weights.bin`) to produce a single contiguous memory image (`combined.bin`). The `xxd` tool is used as follows:

```
flash_init.mem: combined.bin
xxd -p -c 1 combined.bin > flash_init.mem
```

The `-p` flag outputs the binary as a plain hexadecimal stream without addresses or formatting. The `-c 1` flag enforces that each byte is printed on a separate line. This format is required when using `$readmemh` in the simulated flash controller. During simulation, `flash_init.mem` is loaded automatically at startup.

5.2 Image and Weight Generation

The image and weight binaries are generated using a Python script. In this script, the MNIST dataset is loaded using TensorFlow Keras utilities:

```
print("Loading MNIST dataset...")
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

The test dataset is normalized by scaling each pixel value from an integer range `[0, 255]` down to a floating-point range `[0.0, 1.0]`:

```
x_test = x_test.astype("float32") / 255.0
```

The first image from the test set is selected:

MNIST Test Image (Label: 7)

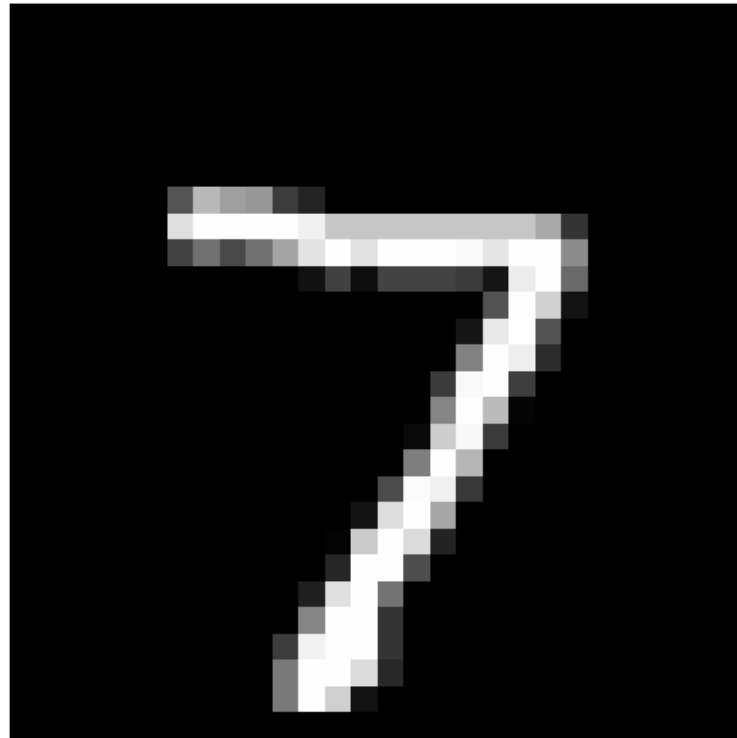


Figure 3: Grayscale Test Image

```
img_arr = x_test[0] # Image of a handdrawn 7
```

Each pixel intensity is then quantized to a 16-bit Q8.8 fixed-point format. Q8.8 is a binary fixed-point representation where:

- The upper 8 bits represent the integer part.
- The lower 8 bits represent the fractional part.
- For example, a real value 0.5 would be stored as 0x0080, since $0.5 \times 256 = 128$.

The dynamic range is approximately $[-128, 127.996]$ with a precision step of $1/256 \approx 0.0039$. The first image from the test set is selected:

```
img_arr = x_test[0] # Image of a handdrawn 7
```

Each pixel intensity is then quantized to a 16-bit Q8.8 fixed-point format. Q8.8 is a binary fixed-point representation where:

- The upper 8 bits represent the integer part.
- The lower 8 bits represent the fractional part.
- For example, a real value 0.5 would be stored as 0x0080, since $0.5 \times 256 = 128$.

- The dynamic range is approximately $[-128, 127.996]$ with a precision step of $1/256 \approx 0.0039$.
-

The conversion function used is:

```
def to_fixed_hex(x):
    fixed = int(round(x * 256))
    fixed &= 0xFFFF
    return format(fixed, "04x")
```

This function multiplies the floating-point value by 256, rounds to the nearest integer, clamps it to 16 bits, and formats it as a 4-character hexadecimal string. The pixel values are written sequentially into a text file (image.hex):

```
test_img_flat = img_arr.flatten()
with open("image.hex", "w") as f:
    for px in test_img_flat:
        f.write(to_fixed_hex(px) + "\n")
```

Each line of image.hex represents one 16-bit Q8.8 encoded pixel, ready for loading into SRAM.

For the model, a feed-forward neural network is created using TensorFlow:

The script builds, trains, and exports a neural network designed for MNIST digit classification:

```
model = Sequential(
    [
        Input(shape=(28, 28)),
        Flatten(),
        Dense(64, activation="relu"),
        Dense(32, activation="relu"),
        Dense(10, activation="linear"),
    ]
)
```

The model consists of:

- A Flatten layer that reshapes the 28x28 image into a 784-element vector.
- A first Dense layer with 64 neurons and ReLU activation. This layer learns 785 parameters per neuron (784 weights + 1 bias).
- A second Dense layer with 32 neurons and ReLU activation. Each neuron has 65 parameters (64 inputs + 1 bias).

- A final Dense layer with 10 output neurons and linear activation. Each output neuron has 33 parameters (32 inputs + 1 bias).

The model is trained using the Adam(adaptive motion) optimizer and uses sparse categorical cross-entropy loss function:

```
model.compile(
    optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
model.fit(x_train, y_train, epochs=5, validation_split=0.1, verbose=1)
```

The model uses the Sparse Categorical Cross-Entropy loss function, which measures the difference between the predicted logits and the correct class label. The loss is minimized using the Adam optimizer, a variant of stochastic gradient descent that adaptively adjusts learning rates per parameter using first and second moment estimates of gradients. During each training iteration, gradients of the loss with respect to every weight and bias are computed using backpropagation. These gradients tell the optimizer how to update each parameter to reduce prediction error. This process repeats over 5 epochs, where each epoch means the model has seen the entire dataset once. A small portion of the data (10%) is held out as validation to monitor performance on unseen samples during training.

Once training completes, the weights and biases are extracted layer-by-layer. Each neuron's bias and weights are combined into a row:

```
def process_layer(layer):
    w, b = (
        layer.get_weights()
    ) # w shape: (in_features, out_features) b shape: (out_features,)
    w = w.T # each row corresponds to one neuron
    rows = []
    for i in range(w.shape[0]):
        row = [b[i]] + list(w[i])
        rows.append(row)
    return np.array(rows)
```

Here, the weight matrices are transposed ($w = w.T$) to match the CPU's expected memory order, where weights for each neuron are stored contiguously after its bias.

Each bias and weight is then quantized into 16-bit Q8.8 format using the same `to_fixed_hex` function as the image quantization. The final export loop sequentially writes the fixed-point parameters to a text file (weights.hex):

```
# Total expected count: 64*785 + 32*65 + 10*33 = 52650.
with open("weights.hex", "w") as f:
    for hex_val in fixed_weights:
        f.write(hex_val + "\n")
```

The output order is strictly neuron-by-neuron. The number of entries for the computation are:

- $64 \text{ neurons} \times 785 \text{ entries} = 50,240 \text{ entries}$ for the first layer.
- $32 \text{ neurons} \times 65 \text{ entries} = 2,080 \text{ entries}$ for the second layer.
- $10 \text{ neurons} \times 33 \text{ entries} = 330 \text{ entries}$ for the third layer.

The total number of 16-bit words is 52,650. These values are later packed into binary format to be concatenated into the final flash initialization image.

5. Toolchain Used

- Vivado
 - I used Vivado to synthesize my hardware design, run functional and timing simulations, and generate the FPGA bitstream for deployment onto the Basys3 development board. Vivado was also used to assign physical FPGA pins in my xdc file, analyze resource usage (such as LUTs and flip-flops), and verify correct timing.
- RISC-V GNU Toolchain
 - <https://formulae.brew.sh/formula/riscv64-elf-gcc>
 - <https://github.com/riscv-collab/riscv-gnu-toolchain>

6. Memory Map

6.1 Overall 32-bit Address Space Map (Von Neumann Architecture)

<u>Address Range</u>	<u>Description</u>
0x0000_0000 – 0x000F_FFFF	SRAM (Internal Memory)
0x0010_0000 – 0x3FFF_FFFF	Unused (Reserved)
0x4000_0000 – 0x4000_0FFF	MMIO (Memory-Mapped I/O)
0x4000_1000 – 0x7FFF_FFFF	Unused (Reserved)
0x8000_0000 – 0x800F_FFFF	Flash Memory
0x8010_0000 – 0xFFFF_FFFF	Unused (Reserved)

6.2 SRAM Layout (0x0000_0000 – 0x000F_FFFF)

Address Range	Purpose
---------------	---------

Address Range	Purpose
0x0001_0000 – 0x0001_30FF	Image Data (MNIST Input)
0x0002_0000 – 0x0002_7FFF	Layer 1 Weights
0x0003_0000 – 0x0003_3FFF	Layer 2 Weights
0x0004_0000 – 0x0004_0FFF	Layer 3 Weights
Remaining SRAM Space	Unused / Reserved

6.3 MMIO Address Map (0x4000_0000 – 0x4000_0FFF)

Address	Function
0x4000_0000	Accelerator Control Register (write '1' to start)
0x4000_0004	Accelerator Output Register (read prediction)
0x4000_0010	Seven-Segment Display Register (write prediction digit)
0x4000_0020	UART Transmit Register (send character via UART)

6.4 Flash Memory Layout (0x8000_0000 – 0x800F_FFFF)

Address Range	Contents
0x8000_0000 – 0x8000_00DF	Firmware Code (Boot Program)
0x8000_00E0 – 0x8000_08FF	Image Data (copy to SRAM at boot)
0x8000_0900 – 0x8005_FFFF	Layer 1, 2, 3 Weights (copy to SRAM at boot)
Remaining Flash Space	Unused / Reserved

7. Test Bench and Validation

7.1 Test Bench Description

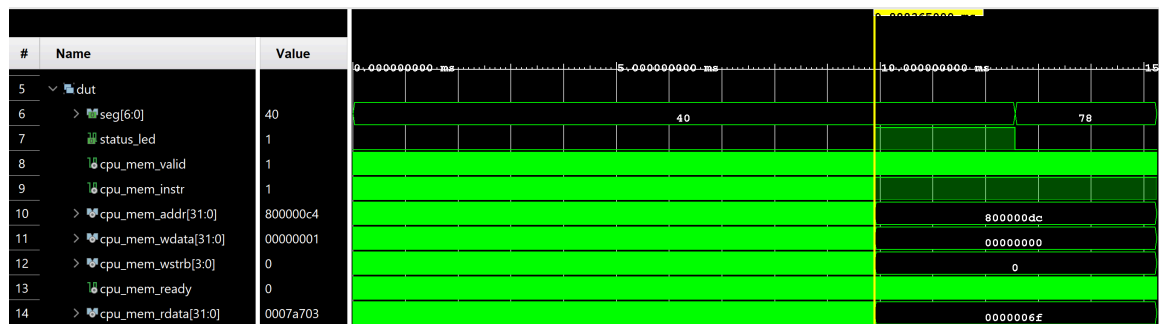


Figure 4: Top Level Testbench Overview

In the top-level simulation of the SoC, the system initially boots and the CPU begins by copying image and weight data from flash memory into SRAM. During this phase,

cpu_mem_addr increments through the flash memory address space (i.e. around 0x800000c4), with cpu_mem_ready driving the read/write handshaking.

After completing the data transfer, the CPU writes to the accelerator control register, triggering the start of neural network inferencing. You can see this in the simulation waveform when the status_led signal transitions to high, indicating that the neural network accelerator is active (nn_busy = 1).

During the inference phase, the accelerator loads image and weight data from SRAM, processes through its internal layers (loading, MAC operations, activation functions), and then produces a prediction result. Once inference is done, the status_led gets driven to low, and the predicted digit is output onto the seven-segment display. In the figure shown above, the seven segment output changes from 0x40 (displaying '4') to 0x78 (displaying '7'), corresponding to the neural network's prediction of the input image.

7.2 Simulation Results

7.2.1 Program Initialization

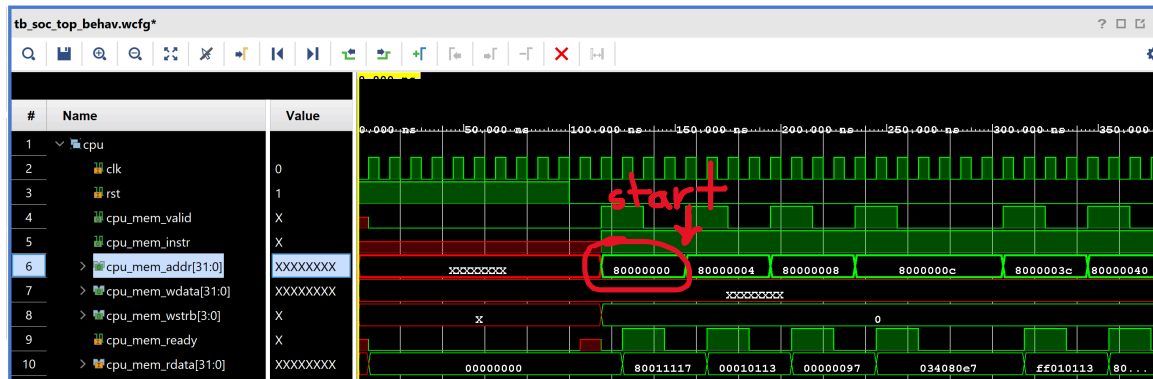


Figure 5: CPU Program Boot

The waveform shows the PicoRV32 CPU beginning execution after reset goes from high to low. The program counter starts at 0x80000000, with cpu_mem_valid and cpu_mem_instr asserted to fetch instructions from flash. The addresses 0x80000000, 0x80000004, and 0x80000008 correspond to the initial instructions of the _start routine. Flash responds with valid instruction words on cpu_mem_rdata, including 0x80011117, 0x00010113, and 0x00000097, matching the disassembled assembly file. Instruction fetches are processed sequentially, which is the same as the expected order in the assembly file.

7.2.2 Image and Weight Copy from Flash to Sram

The following 4 waveforms show execution of the copy_memory function from main(), which transfers data from flash to SRAM. Initially, the CPU reads from flash starting at address 0x800000e0, which is the first address beyond the firmware text section, and writes to 0x00010000, the defined base address for image data. cpu_mem_instr is low during this phase, indicating data access rather than instruction fetch. Subsequent memory writes appear at 0x00020000, 0x00030000, and 0x00040000, matching the

defined base addresses for weight1, weight2, and weight3 data. In each transfer, flash read addresses increment sequentially by 4 bytes while SRAM write addresses increment by 4 bytes also. Valid 32 bit writes happen when `cpu_mem_wstrb = 0xf`. Data being written to SRAM (`cpu_mem_wdata`) and acknowledged by `cpu_mem_ready` reflect that SRAM write cycles are proceeding without stalls in simulation.

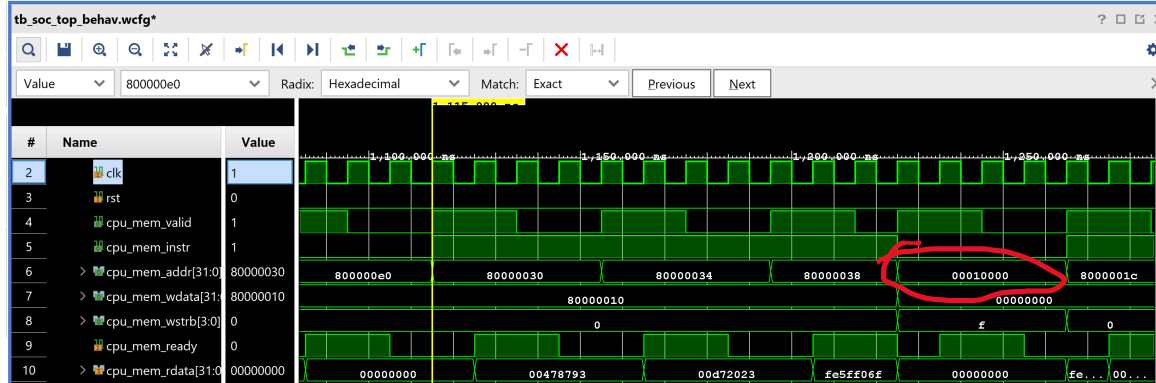


Figure 6: CPU starts to write image data from Flash to SRAM

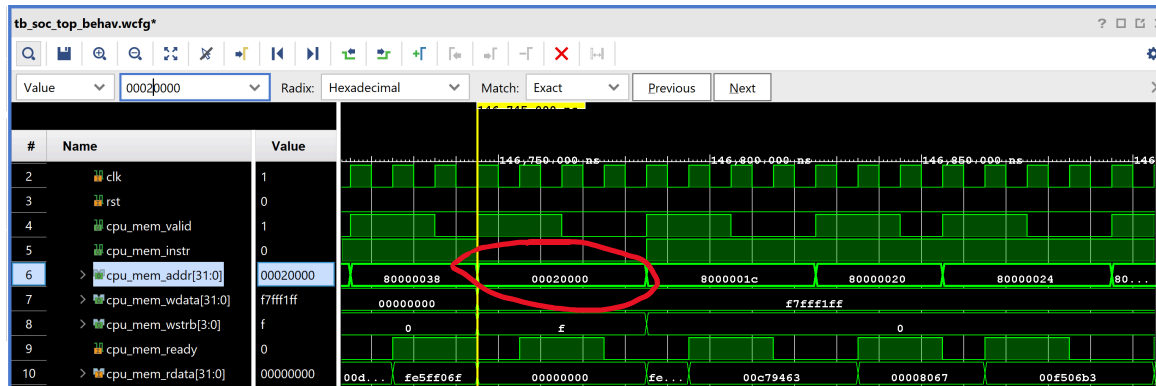


Figure 7: CPU starts to write 1st layer weights from flash to SRAM

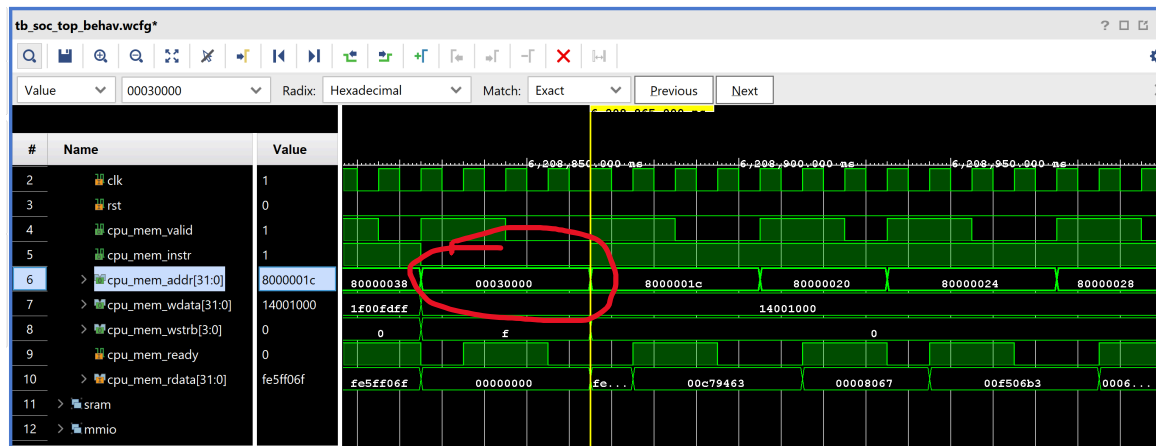


Figure 8: CPU starts to write 2nd layer weights from flash to SRAM

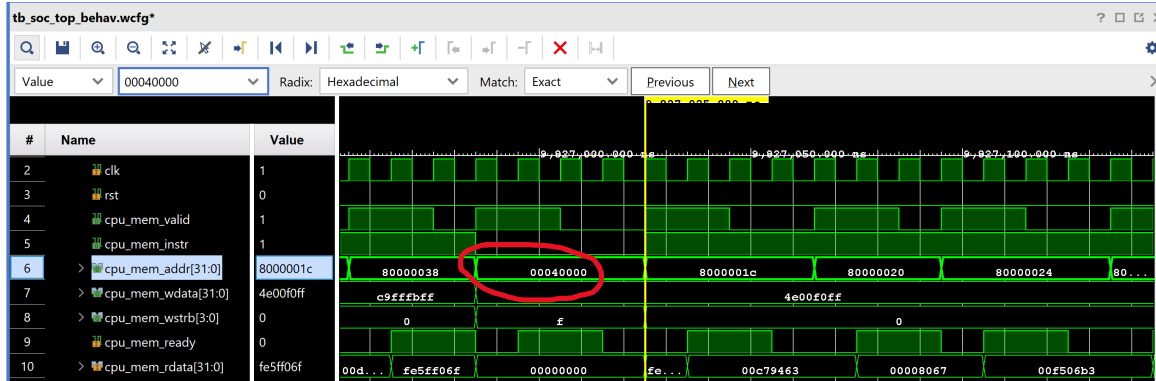


Figure 9: CPU starts to read 3rd layer weights from flash to SRAM

7.2.3 Neural Network Inferencer Starts

The CPU writes 0x00000001 to address 0x40000000, which triggers the accelerator control register (ACCEL_CTRL). `cpu_mem_wstrb = 0xf` confirms a full 32-bit write. This triggers a one-cycle pulse on `mmio_start`, initiating execution in the neural network accelerator. Shortly after the write, the `nn_busy` signal asserts, indicating that the accelerator has entered its active state and begun processing. This sequence confirms proper handoff from the CPU to the accelerator through the MMIO interface.

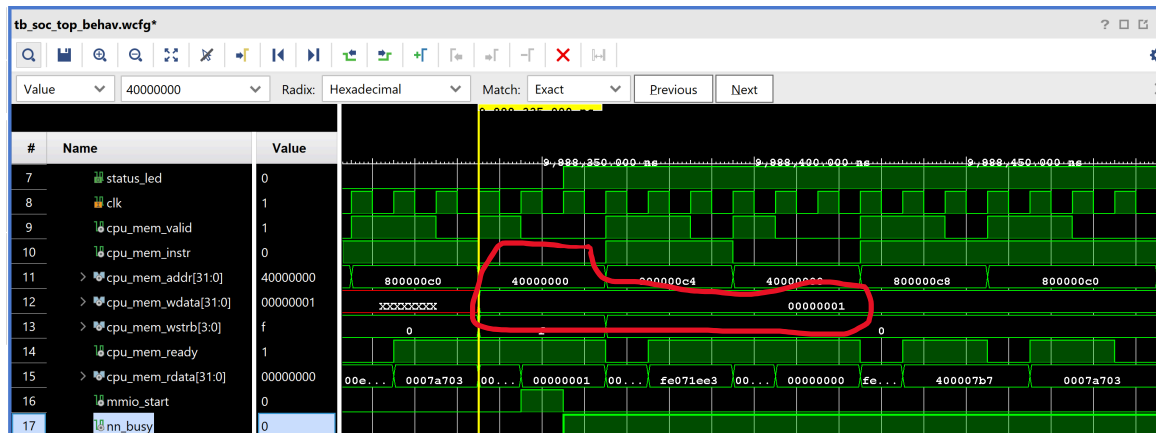


Figure 10: `mmio_start` is triggered and neural network inferencing starts

7.2.4 Neural Network Module Starts Reading Image Data

The neural network accelerator begins reading image data from SRAM at address 0x00010000. The `acc_sram_addr` signal increments in steps of 2 for 16-bit word access. The `acc_sram_data` signal shows pixel values being read, including nonzero entries like 0x0054. Each read cycle uses a 2-cycle delay to safely read data from the SRAM controller.

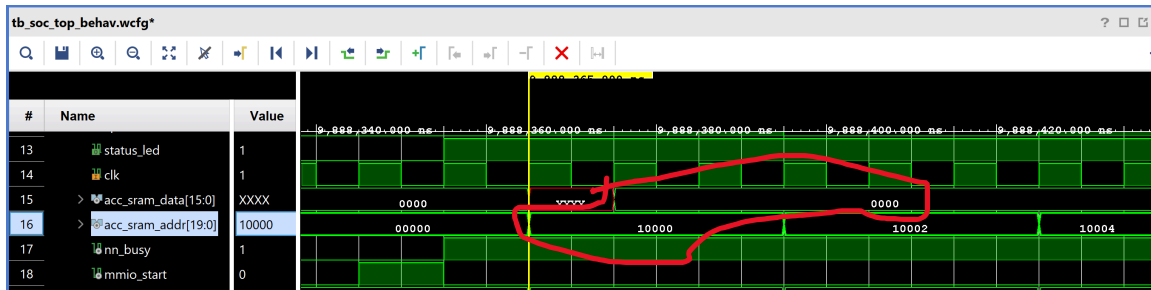


Figure 11: Neural Network Starts Inferencing Image Data

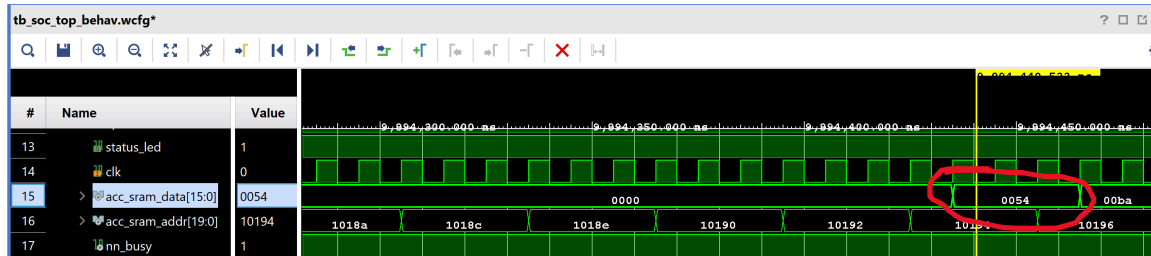


Figure 12: First Non-Zero Image Data

7.2.5 Layer 1 Weight Access

The accelerator reads weights and biases for Layer 1 starting at address 0x00020000. The waveform shows a sequence of reads from incrementing addresses such as 0x20000, 0x20002, and 0x20004. Data values like 0x1061 and 0xFF1 are returned on acc_sram_data.

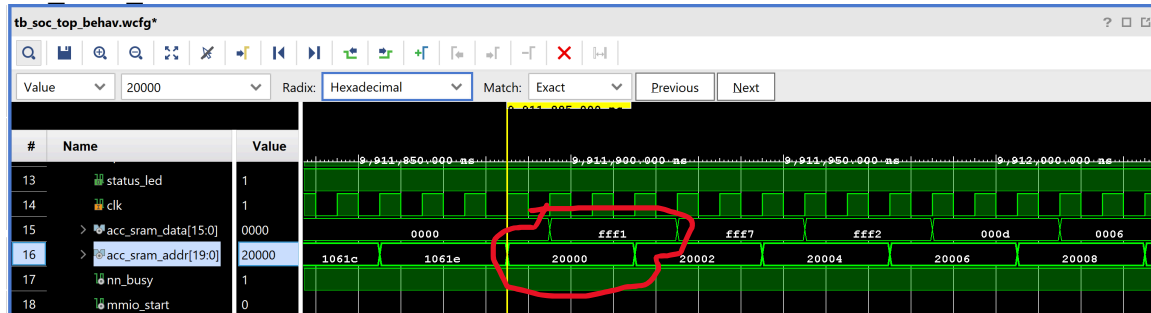


Figure 13: Neural Network Starts Inferencing Layer 1 Weights

7.2.6 Layer 2 Weight Access

The Layer 2 computation begins with SRAM reads starting at 0x00030000. The accelerator reads biases and weights for each of the 32 neurons in this layer. The acc_sram_addr increments in 2-byte intervals, and values such as 0x0016 and 0x0028 are returned with a two-cycle delay.

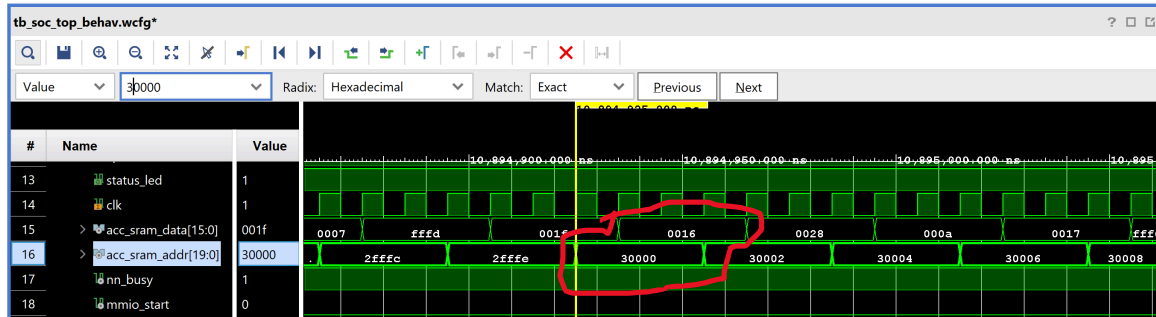


Figure 14: Neural Network Starts Inferencing Layer 2 Weights

7.2.7 Layer 3 Weight Access

This waveform corresponds to Layer 3 weight access, beginning at address 0x00040000. The acc_sram_addr signal advances through sequential addresses and signed 16-bit data like 0xFFC9 and 0x004E are captured on acc_sram_data. These weights are used to compute the final logits from Layer 2 activations prior to performing argmax classification.

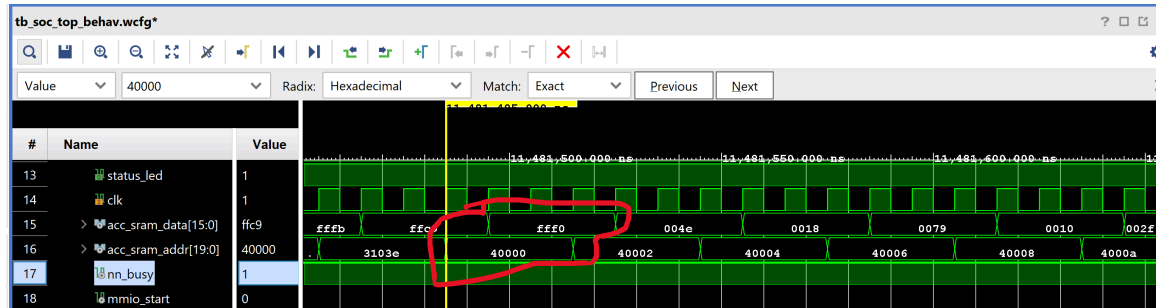


Figure 15: Neural Network Starts Inferencing Layer 3 Weights

7.2.8 Argmax Phase (after Layer 3 Completion)

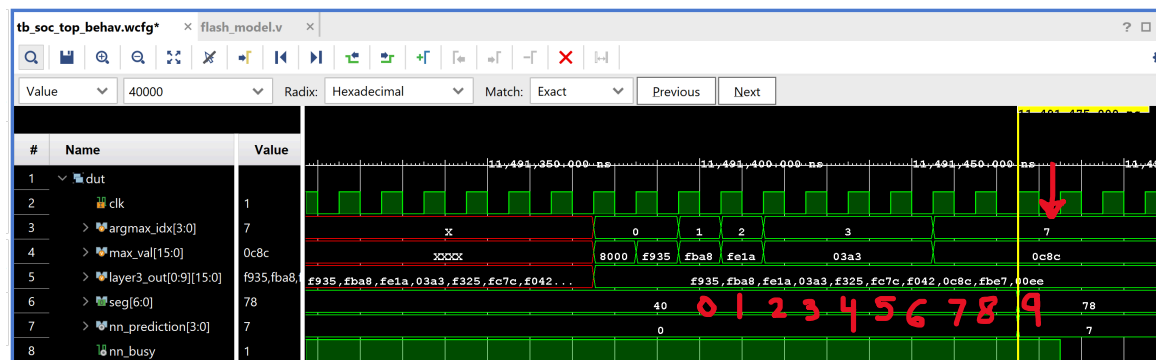


Figure 16: Neural Network Final Phase(Argmax)

After all Layer 3 MAC operations complete, the accelerator transitions into the FSM_ARGMAX state. In this state, the accelerator scans through all 10 output logits stored in the layer3_out array. These values represent raw scores (logits) for each digit class (0–9) and are compared sequentially to determine the maximum.

The signal `argmax_idx` increments from 0 to 9, selecting each neuron output for comparison. The `max_val` register tracks the highest observed value and is updated whenever a larger output is read. Once the scan finishes, the index corresponding to the largest score is written to the `nn_prediction` register. In the waveform, the accelerator identifies neuron 7 as having the maximum output value of `0x0c8c`, resulting in 7 for the predicted digit.

The values shown on the `layer3_out` bus are 16-bit **signed** fixed-point numbers in Q8.8 format. For example:

- $0x0c8c = 3212$ in decimal $= 3212 \div 256 \approx 12.55$
- $0xf935 = -1739$ in decimal $= -1739 \div 256 \approx -6.79$
- $0xfe1a = -486$ in decimal ≈ -1.90

These logits represent confidence scores, where higher values indicate stronger confidence for a particular digit class. In this case, neuron 7 had the highest score (~ 12.55), making it the predicted class.

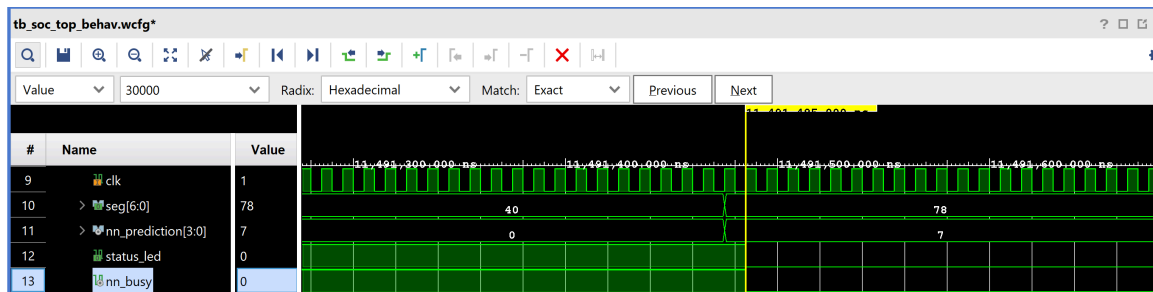


Figure 17: Prediction is Written to 7 Segment Display

7.3 Project Limitations and Bugs

One limitation was the slow transfer of data from flash to SRAM using the PicoRV32 CPU. The CPU reads each 32-bit word from flash through an MMIO interface and writes it to SRAM through another MMIO path. Because the PicoRV32 lacks caching and handles each transfer with a full valid and ready handshake, the entire process is sequential and slow. The flash controller also rebuilds each word from individual bytes, adding more latency.

I also did not implement arbitration in the SRAM controller. The controller grants access to either the CPU or the neural network accelerator depending on which side is active. The accelerator is prioritized by checking if its valid signal is high. This setup works for my current flow where the CPU finishes loading data before the accelerator runs. However, it does not allow concurrent access or time sharing.

The flash controller used in simulation always responds immediately and never models timing delays or real flash behavior. Writes are accepted but do not change the memory contents. I also didn't include support for programming or modifying flash content dynamically.

In the neural network accelerator, I implemented a sequential control flow. Each stage(image load, layer computation, argmax) runs one after another. I did not implement pipelining between stages or overlapping of memory fetch and compute. WAIT_CYCLES is fixed and applies uniformly, regardless of SRAM speed.

The UART interface is included in the design but not used in the inference path. I didn't connect it to the SRAM or use it to trigger inference from external data. I also didn't implement interrupt handling in the CPU. All control is polling-based, which limits responsiveness and prevents asynchronous interaction. These are features I plan to explore to improve my design in the future.

7.4 Worst Case Analysis

The spreadsheet [here](#) includes the worst case loading analysis, timing analysis, noise margin and the memory read cycle timing analysis.

8. Project Development Log

<https://docs.google.com/spreadsheets/d/18wEgSS-ZyXSkXbEI9qJ3WOGOb7FKsOe0TRzOiQbKek/edit?usp=sharing>

9. Special Sources or IP Used

9.1 PicoRV32

For my CPU I used the open-source PicoRV32 core, configured to support the RV32IMC instruction set with compressed instructions enabled. The picorv32_wrapper module instantiates the PicoRV32 core with a program start address of 0x80000000 and connects to a valid-ready native memory interface, exposing memory access signals (mem_valid, mem_ready, mem_addr, mem_wdata, mem_rdata, mem_wstrb) and a trap signal for detecting program halts or faults. The CPU core operates with a standard synchronous design, using a clock and an active-high reset input, and supports external interrupt input through the irq signal. The memory interface allows single read or write operations per cycle based on handshake signals and byte enables, with support for instruction fetch and data operations.

The github repository for this special IP used in this project for the PicoRV32 core and is from [YosysHQ/picorv32 GitHub repository](#).

10. Improvements

To improve performance, I could pipeline the neural network accelerator so that image loading and layer computations overlap instead of running strictly sequentially. This would increase throughput and make better use of available memory bandwidth. I could also design a dedicated memory copier module to transfer data from flash to SRAM independently of the CPU. This would eliminate the bottleneck caused by the PicoRV32's sequential memory accesses. Adding proper arbitration in the SRAM controller would allow the CPU and accelerator to access memory concurrently, which is currently not possible. I also plan to integrate the UART interface to load the image into SRAM.

11. Conclusion

This was my first large project written in Verilog, and it gave me a deep understanding of how hardware systems are built from the ground up. I learned how a CPU works in hardware and how it controls the entire system by issuing memory and peripheral accesses. Using the PicoRV32 RISC-V soft-core processor, I built a working system where the CPU initializes memory, starts computation, and reads back the result. I also learned how to use the RISC-V GNU toolchain to compile C code into a binary firmware image and how to link and locate my code using a custom linker script. I used the toolchain to disassemble my compiled binary into human-readable RISC-V assembly, which helped me debug instruction flow during simulation. Throughout this project, I built several modules including an SRAM controller, a flash memory interface, and a custom neural network accelerator. I simulated each module in my system and verified that the logic executed correctly. Completing this project gave me confidence in hardware-software co-design, and I now feel more comfortable working with toolchains, simulation environments, and hardware integration for FPGAs.

12. References

1. Waterman, A., & Asanović, K. (2021). *The RISC-V Reader: An Open Architecture Atlas* (2nd ed.). RISC-V International.
<https://riscv-programming.org/book.html>
2. Clifford Wolf et al. *PicoRV32 - A Size-Optimized RISC-V CPU*. GitHub repository, <https://github.com/YosysHQ/picorv32>.
3. El Bouazzaoui, A., Hadjoudja, A., & Mouhib, O. (2024). *Real-Time Adaptive Neural Network on FPGA: Enhancing Adaptability through Dynamic Classifier Selection*. arXiv preprint arXiv:2311.09516. <https://arxiv.org/abs/2311.09516>
4. Antunes, P., & Podobas, A. (2025). *FPGA-Based Neural Network Accelerators for Space Applications: A Survey*. arXiv preprint arXiv:2504.16173.
<https://arxiv.org/abs/2504.16173>
5. COMPE475 Class Lectures
6. AMD. (2024). *Artix-7 FPGAs data sheet: DC and AC switching characteristics (DS181, v1.27.1)*. https://docs.amd.com/v/u/en-US/ds181_Artix_7_Data_Sheet
7. Infineon Technologies. (2020). *CY62167G/CY62167GE MoBL 16-Mbit static RAM with error-correcting code (ECC) (Rev. 19.00)* [Data sheet].
[https://www.infineon.com/dgdl/Infineon-CY62167G_CY62167GE_MOBL_16_MBIT_\(1M_WORDS_X_16_BIT_2M_WORDS_X_8_BIT\)_STATIC_RAM_WITH_ERROR-CORRECTING_CODE_\(ECC\)-DataSheet-v19_00-EN.pdf?fileId=8ac78c8c7d0d8da4017d0eccbe004700&utm_source=cypress&utm_medium=referral](https://www.infineon.com/dgdl/Infineon-CY62167G_CY62167GE_MOBL_16_MBIT_(1M_WORDS_X_16_BIT_2M_WORDS_X_8_BIT)_STATIC_RAM_WITH_ERROR-CORRECTING_CODE_(ECC)-DataSheet-v19_00-EN.pdf?fileId=8ac78c8c7d0d8da4017d0eccbe004700&utm_source=cypress&utm_medium=referral)
8. Infineon Technologies. (2018). *S29AL016J 16-Mbit (2M × 8-bit/1M × 16-bit) 3 V boot sector flash memory (Rev. 18.00)* [Data sheet].
[https://www.infineon.com/dgdl/Infineon-S29AL016J_16_MBIT_\(2M_X_8_BIT](https://www.infineon.com/dgdl/Infineon-S29AL016J_16_MBIT_(2M_X_8_BIT)

13. Source Code

13.1 soc_top.v

```
`timescale 1ns/1ps
module soc_top(
    input      clk,
    input      rst,
    input      rx_serial,
    output     tx_serial,
    output [6:0] seg,      // Seven segment display segments driven by NN
    prediction
    output [3:0] an,      // Digit enables for seven seg display
    output     status_led // Status LED (shows accelerator busy)
);

    // CPU (PicoRV32) Memory Interface Signals
    wire      cpu_mem_valid;
    wire      cpu_mem_instr;
    wire [31:0] cpu_mem_addr;
    wire [31:0] cpu_mem_wdata;
    wire [3:0]  cpu_mem_wstrb;
    wire      cpu_mem_ready;
    wire [31:0] cpu_mem_rdata;
    wire [31:0] trap;
    wire [31:0] irq;

    assign irq = 32'b0;

    picorv32_wrapper cpu_inst (
        .clk(clk),
        .rst(rst),
        .cpu_mem_valid(cpu_mem_valid),
        .cpu_mem_instr(cpu_mem_instr),
        .cpu_mem_addr(cpu_mem_addr),
        .cpu_mem_wdata(cpu_mem_wdata),
        .cpu_mem_wstrb(cpu_mem_wstrb),
        .cpu_mem_ready(cpu_mem_ready),
        .cpu_mem_rdata(cpu_mem_rdata),
        .trap(trap),
        .irq(irq)
    );
```

```

// Address Decode
// - Flash: addresses with top nibble 0x8
// - MMIO: addresses with top nibble 0x4
// - SRAM: all others
wire flash_sel = (cpu_mem_addr[31:28] == 4'h8);
wire mmio_sel  = (cpu_mem_addr[31:28] == 4'h4);
wire sram_sel  = ~(flash_sel | mmio_sel);

wire [31:0] flash_rdata, mmio_rdata, sram_rdata;
wire        flash_ready, mmio_ready, sram_ready;

flash_controller flash_inst (
    .clk(clk),
    .rst(rst),
    .mmio_valid(cpu_mem_valid & flash_sel),
    .mmio_write(|cpu_mem_wstrb & flash_sel),
    .mmio_addr(cpu_mem_addr),
    .mmio_wdata(cpu_mem_wdata),
    .mmio_rdata(flash_rdata),
    .mmio_ready(flash_ready),
    // External flash connections - in future will tie to board level signals
    .flash_addr(),
    .flash_data(),
    .flash_BYTE(),
    .flash_CE(),
    .flash_OE(),
    .flash_WE(),
    .flash_WP(),
    .flash_RESET(),
    .flash_RYBY(1'b1) // For simulation purposes, flash is always ready
);

// MMIO module (Drives other peripheral)
// The MMIO module drives control signals (accelerator start) and provides status
reporting
// mmio_start output triggers the NN accelerator
wire mmio_start;
mmio mmio_inst (
    .clk(clk),
    .rst(rst),
    .mmio_valid(cpu_mem_valid & mmio_sel),
    .mmio_write(|cpu_mem_wstrb & mmio_sel),
    .mmio_addr(cpu_mem_addr),
    .mmio_wdata(cpu_mem_wdata),
    .mmio_rdata(mmio_rdata),

```

```

        .mmio_ready(mmio_ready),
        .status_led(),
        .mmio_start(mmio_start)
    );

    // For the CPU, the relevant address bits are mapped into the SRAM region.
    wire [19:0] cpu_sram_addr = cpu_mem_addr[19:0];

    // These wires come from the SRAM controller arbitration for the NN side.
    wire [19:0] acc_sram_addr;
    wire [15:0] acc_sram_data;

    // Single port SRAM controller that arbitrates between the CPU and NN accelerator.
    // The simple arbitration logic gives priority to the NN accelerator
    // when it is busy.
    wire nn_busy;

    sram_controller sram_inst (
        .clk(clk),
        .rst(rst),
        // CPU interface
        .cpu_valid(cpu_mem_valid & sram_sel),
        .cpu_write(!cpu_mem_wstrb & sram_sel),
        .cpu_addr(cpu_sram_addr),
        .cpu_wdata(cpu_mem_wdata),
        .cpu_rdata(sram_rdata),
        .cpu_ready(sram_ready),
        // NN accelerator interface
        .nn_valid(nn_busy), // NN accelerator busy flag indicates active use
        .nn_write(1'b0),   // NN accelerator performs read-only accesses for now
        .nn_addr(acc_sram_addr),
        .nn_data(acc_sram_data),
        // External single-port SRAM connections
        .sram_addr(),      // In future will be connected to physical SRAM
        .sram_data(),
        .sram_CE(),
        .sram_OE(),
        .sram_WE(),
        .sram_BHE(),
        .sram_BLE(),
        .sram_BYTE(),
        .sram_WP(),
        .sram_RESET(),
        .sram_RYBY(1'b1)
    );

```

```

// Instantiate Neural Network Accelerator
// (uses shared SRAM access via the controller)
wire [3:0] nn_prediction;
nn_accelerator nn_acc_inst (
    .clk(clk),
    .rst(rst),
    .start(mmio_start),      // Triggered by MMIO register write
    .busy(nn_busy),
    .prediction(nn_prediction),
    .sram_addr(acc_sram_addr),
    .sram_data(acc_sram_data),
    .weight1_base(32'h00020000),
    .weight2_base(32'h00030000),
    .weight3_base(32'h00040000),
    .image_base(32'h00010000)
);

wire [6:0] seg_out;
wire [3:0] an_out;
seven_seg seven_seg_inst (
    .digit(nn_prediction),
    .seg(seg_out),
    .an(an_out)
);

assign seg = seg_out;
assign an = an_out;

assign status_led = nn_busy;

// Bus Multiplexer for CPU reads
assign cpu_mem_rdata = flash_sel ? flash_rdata :
                        mmio_sel ? mmio_rdata :
                        sram_rdata;
assign cpu_mem_ready = flash_sel ? flash_ready :
                        mmio_sel ? mmio_ready :
                        sram_ready;

// For simulation, tie UART TX to idle (high)
assign tx_serial = 1'b1;

endmodule

```

13.2 picorv32_wrapper.v

```
`timescale 1ns / 1ps
```

```

module picorv32_wrapper(
    input      clk,
    input      rst,          // Active-high reset
    output     cpu_mem_valid,
    output     cpu_mem_instr,
    output [31:0] cpu_mem_addr,
    output [31:0] cpu_mem_wdata,
    output [3:0] cpu_mem_wstrb,
    input      cpu_mem_ready,
    input [31:0] cpu_mem_rdata,
    output [31:0] trap,
    input [31:0] irq
);

picorv32 #(
    .PROGADDR_RESET(32'h80000000),
    .COMPRESSED_ISA(1'b1)
) picorv32_inst (
    .clk          (clk),
    .resetn       (~rst),
    .trap         (trap),
    .mem_valid    (cpu_mem_valid),
    .mem_instr    (cpu_mem_instr),
    .mem_ready    (cpu_mem_ready),
    .mem_addr     (cpu_mem_addr),
    .mem_wdata    (cpu_mem_wdata),
    .mem_wstrb    (cpu_mem_wstrb),
    .mem_rdata    (cpu_mem_rdata),
    .irq          (irq)
);

endmodule

```

13.3 *flash_controller.v*

```

`timescale 1ns/1ps

module flash_controller(
    input      clk,
    input      rst,
    input      mmio_valid,
    input      mmio_write,
    input [31:0] mmio_addr,
    input [31:0] mmio_wdata,
    output reg [31:0] mmio_rdata,
    output reg      mmio_ready,
    output [19:0] flash_addr,

```

```

    inout [7:0] flash_data,
    output      flash_BYTE,
    output      flash_CE,
    output      flash_OE,
    output      flash_WE,
    output      flash_WP,
    output      flash_RESET,
    input       flash_RYBY
);

`ifdef SIMULATION
    parameter MEM_SIZE = 1048576;
    reg [7:0] mem [0:MEM_SIZE-1];

    initial begin
        $readmemh("flash_init.mem", mem);
    end

    assign flash_addr = 20'b0;
    assign flash_data = 8'bz;
    assign flash_BYTE = 1'b0;
    assign flash_CE = 1'b0;
    assign flash_OE = 1'b0;
    assign flash_WE = 1'b1;
    assign flash_WP = 1'b0;
    assign flash_RESET = 1'b1;

    integer offset;
    always @(posedge clk) begin
        if (rst) begin
            mmio_rdata <= 32'b0;
            mmio_ready <= 0;
        end else if (mmio_valid) begin
            mmio_ready <= 1;
            if (mmio_write) begin
                // Read only, just echo writes
                mmio_rdata <= mmio_wdata;
            end else begin
                // Translate 0x80000000 -> offset 0 in mem
                offset = (mmio_addr - 32'h80000000) & 32'hFFFFFFC; // Word align the
address
                // Combine 4 bytes in little endian format
                mmio_rdata <= {mem[offset+3], mem[offset+2], mem[offset+1],
mem[offset]};
                $display("Flash read: addr = 0x%h, offset = %0d, data = %h %h %h %h ->
%h",

```

```

        mmio_addr, offset,
        mem[offset+3], mem[offset+2], mem[offset+1], mem[offset],
        {mem[offset+3], mem[offset+2], mem[offset+1], mem[offset]}});
    end
end else begin
    mmio_ready <= 0;
end
end
`else
    // Synthesis version: no real flash logic
    assign flash_addr = 20'bz;
    assign flash_data = 8'bz;
    assign flash_BYTE = 1'bz;
    assign flash_CE = 1'bz;
    assign flash_OE = 1'bz;
    assign flash_WE = 1'bz;
    assign flash_WP = 1'bz;
    assign flash_RESET = 1'bz;

    always @(posedge clk) begin
        if (rst) begin
            mmio_rdata <= 32'b0;
            mmio_ready <= 0;
        end else if (mmio_valid) begin
            mmio_ready <= 1;
            if (mmio_write)
                mmio_rdata <= mmio_wdata;
            else
                mmio_rdata <= 32'b0;
        end else begin
            mmio_ready <= 0;
        end
    end
end
`endif
endmodule

```

13.3 mmio.v

```

`timescale 1ns/1ps
module mmio(
    input      clk,
    input      rst,
    input      mmio_valid,
    input      mmio_write,
    input [31:0] mmio_addr,
    input [31:0] mmio_wdata,

```

```

output reg [31:0] mmio_rdata,
output reg      mmio_ready,
// Peripheral outputs
output reg      status_led,
// Accelerator control signal output
output reg      mmio_start
);

always @(posedge clk or posedge rst) begin
    if (rst) begin
        mmio_rdata <= 32'b0;
        mmio_ready <= 0;
        mmio_start <= 0;
        status_led <= 0;
    end else begin
        mmio_ready <= 1;
        if (mmio_valid) begin
            if (mmio_write) begin
                // writing to the accelerator control register (offset 0x00)
                if (mmio_addr == 32'h40000000 && mmio_wdata == 1) begin
                    mmio_start <= 1;
                end
                mmio_rdata <= mmio_wdata;
            end else begin
                mmio_rdata <= 32'b0;
            end
        end else begin
            mmio_start <= 0;
        end
    end
end
endmodule

```

13.4 sram.v

```

`timescale 1ns/1ps
module sram_controller(
    input      clk,
    input      rst,
    // CPU interface signals
    input      cpu_valid,
    input      cpu_write,
    input [19:0] cpu_addr,
    input [31:0] cpu_wdata,
    output [31:0] cpu_rdata,
    output      cpu_ready,

```

```

    // Neural Network accelerator interface signals
    input      nn_valid,
    input      nn_write,
    input [19:0] nn_addr,
    output [15:0] nn_data,
    // External single port SRAM interface
    output [19:0] sram_addr,
    inout [15:0] sram_data,
    output      sram_CE,
    output      sram_OE,
    output      sram_WE,
    output      sram_BHE,
    output      sram_BLE,
    output      sram_BYTE,
    output      sram_WP,
    output      sram_RESET,
    input      sram_RYBY
);

`ifdef SIMULATION
    // Simulation model: a simple 1MB (1048576 word) SRAM array, where each location
    is 16 bits
    parameter MEM_SIZE = 1048576;
    reg [15:0] mem [0:MEM_SIZE-1];

    // Registers to drive CPU/NN outputs
    reg [31:0] cpu_rdata_reg;
    reg      cpu_ready_reg;
    reg [15:0] nn_data_reg;

    // The 20 bit addresses provided by the CPU and the NN accelerator are byte
    addresses
    // To index our 16-bit words in "mem", we divide by 2 (i.e. shift right by 1)
    wire [18:0] cpu_index = cpu_addr >> 1;
    wire [18:0] nn_index  = nn_addr  >> 1;

    // For simulation, drive the external SRAM interface with constant signals
    assign sram_addr = nn_valid ? nn_addr : cpu_addr;
    assign sram_CE   = 1'b0;
    assign sram_OE   = 1'b0;
    assign sram_WE   = 1'b1;
    assign sram_BHE  = 1'b0;
    assign sram_BLE  = 1'b0;
    assign sram_BYTE = 1'b0;
    assign sram_WP   = 1'b0;
    assign sram_RESET= 1'b1;

```

```

assign sram_data = 16'bz; // Tri state bus

// Always block to simulate memory access
always @(posedge clk) begin
    if (rst) begin
        cpu_rdata_reg <= 32'b0;
        cpu_ready_reg <= 0;
        nn_data_reg    <= 16'b0;
    end else begin
        // --- CPU Interface ---
        if (cpu_valid) begin
            if (cpu_write) begin
                // Writes a 32 bit word to SRAM
                // Since the CPU writes using byte addresses, we convert the address
                // to an index by shifting right by 1
                // Then split the 32 bit word into two 16 bit halves Also, swap the
                // bytes within each half:
                // For 32'hba005400:
                //     Lower half becomes {cpu_wdata[7:0], cpu_wdata[15:8]} =
                // {8'h00, 8'h54} ? 16'h0054
                //     Upper half becomes {cpu_wdata[23:16], cpu_wdata[31:24]} =
                // {8'h00, 8'hba} ? 16'h00ba
                mem[cpu_index]    <= { cpu_wdata[7:0],   cpu_wdata[15:8]   };
                mem[cpu_index + 1] <= { cpu_wdata[23:16], cpu_wdata[31:24] };
                cpu_rdata_reg     <= 32'b0;
            end else begin
                // For a CPU read: combine two 16 bit words
                // The read returns the word as { upper, lower }
                cpu_rdata_reg <= { mem[cpu_index + 1], mem[cpu_index] };
            end
            cpu_ready_reg <= 1;
        end else begin
            cpu_ready_reg <= 0;
        end

        if (nn_valid) begin
            // Convert the NN address to a word index (by dividing by 2) and read
            // the 16 bit word
            nn_data_reg <= mem[nn_index];
        end
    end
end

assign cpu_rdata = cpu_rdata_reg;
assign cpu_ready = cpu_ready_reg;
assign nn_data   = nn_data_reg;

```

```

`else
    // Synthesis version: no real SRAM model, external memory is not implemented
    assign sram_addr = cpu_valid ? cpu_addr : nn_addr;
    assign sram_CE   = 1'bz;
    assign sram_OE   = 1'bz;
    assign sram_WE   = 1'bz;
    assign sram_BHE  = 1'bz;
    assign sram_BLE  = 1'bz;
    assign sram_BYTE = 1'bz;
    assign sram_WP   = 1'bz;
    assign sram_RESET = 1'bz;
    assign cpu_rdata = 32'b0;
    assign cpu_ready = 1'b1;
    assign nn_data   = 16'b0;
`endif

endmodule

```

13.5 nn_accelerator.v

```

`timescale 1ns/1ps
module nn_accelerator(
    input      clk,
    input      rst,
    input      start,
    output reg  busy,
    output reg [3:0] prediction,
    // Dual-port SRAM interface port B (read-only)
    output reg [19:0] sram_addr,
    input      [15:0] sram_data,
    // Base addresses for weights and image stored in SRAM
    input [31:0] weight1_base,
    input [31:0] weight2_base,
    input [31:0] weight3_base,
    input [31:0] image_base
);

    // Network parameters (each row: bias + weights)
    localparam LAYER1_NEURONS = 64;
    localparam LAYER1_INPUTS  = 784;
    localparam LAYER1_ROW_SIZE = 785; // 1 bias + 784 weights

    localparam LAYER2_NEURONS = 32;
    localparam LAYER2_INPUTS  = LAYER1_NEURONS;
    localparam LAYER2_ROW_SIZE = 65;  // 1 bias + 64 weights

```

```

localparam LAYER3_NEURONS = 10;
localparam LAYER3_INPUTS = LAYER2_NEURONS;
localparam LAYER3_ROW_SIZE = 33; // 1 bias + 32 weights

// Internal arrays for image data and layer outputs.
reg [15:0] image [0:LAYER1_INPUTS-1];
reg signed [15:0] layer1_out [0:LAYER1_NEURONS-1];
reg signed [15:0] layer2_out [0:LAYER2_NEURONS-1];
reg signed [15:0] layer3_out [0:LAYER3_NEURONS-1];

// WAIT_CYCLES parameter controls how many cycles to wait before sampling
sram_data.
localparam WAIT_CYCLES = 2;

// FSM state encoding
localparam FSM_IDLE = 5'd0;
localparam FSM_LOAD_IMG = 5'd1;
localparam FSM_LOAD_IMG_WAIT = 5'd2;

localparam FSM_L1_INIT = 5'd3;
localparam FSM_L1_BIAS_WAIT = 5'd4;
localparam FSM_L1_MAC = 5'd5;
localparam FSM_L1_WEIGHT_WAIT = 5'd6;

localparam FSM_L2_INIT = 5'd7;
localparam FSM_L2_BIAS_WAIT = 5'd8;
localparam FSM_L2_MAC = 5'd9;
localparam FSM_L2_WEIGHT_WAIT = 5'd10;

localparam FSM_L3_INIT = 5'd11;
localparam FSM_L3_BIAS_WAIT = 5'd12;
localparam FSM_L3_MAC = 5'd13;
localparam FSM_L3_WEIGHT_WAIT = 5'd14;

localparam FSM_ARGMAX = 5'd15;
localparam FSM_DONE = 5'd16;

reg [4:0] state;
reg [15:0] neuron_cnt;
reg [15:0] input_cnt;
// 32-bit accumulator in Q16.16 format.
reg signed [31:0] accumulator;

// For argmax
reg [3:0] argmax_idx;
reg signed [15:0] max_val;

```

```

// Rounding constant (half of 256)
localparam signed [31:0] ROUND_CONST = 32'd128;

// Wait cycle counter used in wait states
reg [3:0] wait_cnt;

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state      <= FSM_IDLE;
        busy       <= 0;
        prediction  <= 0;
        neuron_cnt  <= 0;
        input_cnt   <= 0;
        accumulator <= 0;
        sram_addr   <= 0;
        wait_cnt    <= 0;
    end else begin
        case (state)
            // --- Image Load Phase ---
            FSM_IDLE: begin
                if (start) begin
                    busy <= 1;
                    neuron_cnt <= 0;
                    input_cnt <= 0;
                    state <= FSM_LOAD_IMG;
                end
            end
            FSM_LOAD_IMG: begin
                // Read image data: multiply input index by 2 (<< 1) to scale to 16-bit
                // addresses.
                sram_addr <= image_base[19:0] + (input_cnt << 1);
                wait_cnt <= 0;
                state <= FSM_LOAD_IMG_WAIT;
            end
            FSM_LOAD_IMG_WAIT: begin
                if (wait_cnt < WAIT_CYCLES-1)
                    wait_cnt <= wait_cnt + 1;
                else begin
                    wait_cnt <= 0;
                    image[input_cnt] <= sram_data;
                    if (input_cnt == (LAYER1_INPUTS - 1)) begin
                        input_cnt <= 0;
                        neuron_cnt <= 0;
                        state <= FSM_L1_INIT;
                    end else begin

```

```

        input_cnt <= input_cnt + 1;
        state <= FSM_LOAD_IMG;
    end
end
end

// --- Layer 1 (ReLU Activation) ---
FSM_L1_INIT: begin
    // Fetch bias for neuron: multiply the offset by 2.
    sram_addr <= weight1_base[19:0] + ((neuron_cnt * LAYER1_ROW_SIZE) <<
1);

    wait_cnt <= 0;
    state <= FSM_L1_BIAS_WAIT;
end
FSM_L1_BIAS_WAIT: begin
    if (wait_cnt < WAIT_CYCLES-1)
        wait_cnt <= wait_cnt + 1;
    else begin
        wait_cnt <= 0;
        // Sample bias and shift it to Q16.16 format.
        accumulator <= $signed(sram_data) <<< 8;
        input_cnt <= 0;
        state <= FSM_L1_MAC;
    end
end
FSM_L1_MAC: begin
    // Fetch the next weight:
    // The address is computed as base + (neuron_cnt * LAYER1_ROW_SIZE +
1 + input_cnt)*2.
    sram_addr <= weight1_base[19:0] + (((neuron_cnt * LAYER1_ROW_SIZE) +
1 + input_cnt) << 1);
    wait_cnt <= 0;
    state <= FSM_L1_WEIGHT_WAIT;
end
FSM_L1_WEIGHT_WAIT: begin
    if (wait_cnt < WAIT_CYCLES-1)
        wait_cnt <= wait_cnt + 1;
    else begin
        wait_cnt <= 0;
        // Multiply image pixel by weight and accumulate the result.
        accumulator <= accumulator + ($signed(image[input_cnt]) *
$signed(sram_data));
        if (input_cnt == (LAYER1_INPUTS - 1)) begin
            // Apply ReLU activation.
            if (accumulator[31])
                layer1_out[neuron_cnt] <= 0;

```

```

        else
            layer1_out[neuron_cnt] <= (accumulator + ROUND_CONST) >>> 8;
        if (neuron_cnt == (LAYER1_NEURONS - 1)) begin
            neuron_cnt <= 0;
            state <= FSM_L2_INIT;
        end else begin
            neuron_cnt <= neuron_cnt + 1;
            state <= FSM_L1_INIT;
        end
        input_cnt <= 0;
    end else begin
        input_cnt <= input_cnt + 1;
        state <= FSM_L1_MAC;
    end
end
end

// --- Layer 2 (ReLU Activation) ---
FSM_L2_INIT: begin
    sram_addr <= weight2_base[19:0] + ((neuron_cnt * LAYER2_ROW_SIZE) <<
1);

    wait_cnt <= 0;
    state <= FSM_L2_BIAS_WAIT;
end
FSM_L2_BIAS_WAIT: begin
    if (wait_cnt < WAIT_CYCLES-1)
        wait_cnt <= wait_cnt + 1;
    else begin
        wait_cnt <= 0;
        accumulator <= $signed(sram_data) <<< 8;
        input_cnt <= 0;
        state <= FSM_L2_MAC;
    end
end
FSM_L2_MAC: begin
    sram_addr <= weight2_base[19:0] + (((neuron_cnt * LAYER2_ROW_SIZE) +
1 + input_cnt) << 1);
    wait_cnt <= 0;
    state <= FSM_L2_WEIGHT_WAIT;
end
FSM_L2_WEIGHT_WAIT: begin
    if (wait_cnt < WAIT_CYCLES-1)
        wait_cnt <= wait_cnt + 1;
    else begin
        wait_cnt <= 0;

```

```

        accumulator <= accumulator + ($signed(layer1_out[input_cnt]) *
$signed(sram_data));
        if (input_cnt == (LAYER2_INPUTS - 1)) begin
            if (accumulator[31])
                layer2_out[neuron_cnt] <= 0;
            else
                layer2_out[neuron_cnt] <= (accumulator + ROUND_CONST) >>> 8;
            if (neuron_cnt == (LAYER2_NEURONS - 1)) begin
                neuron_cnt <= 0;
                state <= FSM_L3_INIT;
            end else begin
                neuron_cnt <= neuron_cnt + 1;
                state <= FSM_L2_INIT;
            end
            input_cnt <= 0;
        end else begin
            input_cnt <= input_cnt + 1;
            state <= FSM_L2_MAC;
        end
    end
end

// --- Layer 3 (Linear Output) ---
FSM_L3_INIT: begin
    sram_addr <= weight3_base[19:0] + ((neuron_cnt * LAYER3_ROW_SIZE) <<
1);

    wait_cnt <= 0;
    state <= FSM_L3_BIAS_WAIT;
end
FSM_L3_BIAS_WAIT: begin
    if (wait_cnt < WAIT_CYCLES-1)
        wait_cnt <= wait_cnt + 1;
    else begin
        wait_cnt <= 0;
        accumulator <= $signed(sram_data) <<< 8;
        input_cnt <= 0;
        state <= FSM_L3_MAC;
    end
end
FSM_L3_MAC: begin
    sram_addr <= weight3_base[19:0] + (((neuron_cnt * LAYER3_ROW_SIZE) +
1 + input_cnt) << 1);
    wait_cnt <= 0;
    state <= FSM_L3_WEIGHT_WAIT;
end
FSM_L3_WEIGHT_WAIT: begin

```

```

        if (wait_cnt < WAIT_CYCLES-1)
            wait_cnt <= wait_cnt + 1;
        else begin
            wait_cnt <= 0;
            accumulator <= accumulator + ($signed(layer2_out[input_cnt]) *
$signed(sram_data));
            if (input_cnt == (LAYER3_INPUTS - 1)) begin
                layer3_out[neuron_cnt] <= (accumulator + ROUND_CONST) >>> 8;
                if (neuron_cnt == (LAYER3_NEURONS - 1)) begin
                    input_cnt <= 0;
                    max_val <= 16'h8000;
                    argmax_idx <= 0;
                    state <= FSM_ARGMAX;
                end else begin
                    neuron_cnt <= neuron_cnt + 1;
                    state <= FSM_L3_INIT;
                end
            end else begin
                input_cnt <= input_cnt + 1;
                state <= FSM_L3_MAC;
            end
        end
    end

    // --- Argmax Calculation ---
    FSM_ARGMAX: begin
        if ($signed(layer3_out[input_cnt]) > $signed(max_val)) begin
            max_val <= layer3_out[input_cnt];
            argmax_idx <= input_cnt[3:0];
        end
        if (input_cnt == (LAYER3_NEURONS - 1)) begin
            prediction <= argmax_idx;
            state <= FSM_DONE;
        end else begin
            input_cnt <= input_cnt + 1;
            state <= FSM_ARGMAX;
        end
    end
    FSM_DONE: begin
        busy <= 0;
        state <= FSM_IDLE;
    end
    default: state <= FSM_IDLE;
endcase
end
end

```

```
endmodule
```

13.6 seven_seg.v

```
`timescale 1ns / 1ps
module seven_seg(
    input [3:0] digit,
    output [6:0] seg, // segments a, b, c, d, e, f, g
    output [3:0] an   // digit enable signals (active low)
);
    reg [6:0] seg_reg;
    reg [3:0] an_reg;

    assign seg = seg_reg;
    assign an = an_reg;

    // Combinational decoder for a common-anode display
    always @(*) begin
        case (digit)
            4'd0: seg_reg = 7'b1000000; // Pattern for 0 (segments a b c d e f on, g
off)
            4'd1: seg_reg = 7'b1111001; // Pattern for 1 (segments b and c on)
            4'd2: seg_reg = 7'b0100100; // Pattern for 2 (segments a, b, d, e, g on)
            4'd3: seg_reg = 7'b0110000; // Pattern for 3 (segments a, b, c, d, g on)
            4'd4: seg_reg = 7'b0011001; // Pattern for 4 (segments b, c, f, g on)
            4'd5: seg_reg = 7'b0010010; // Pattern for 5 (segments a, c, d, f, g on)
            4'd6: seg_reg = 7'b0000010; // Pattern for 6 (segments a, c, d, e, f, g
on)
            4'd7: seg_reg = 7'b1111000; // Pattern for 7 (segments a, b, c on)
            4'd8: seg_reg = 7'b0000000; // Pattern for 8 (all segments on)
            4'd9: seg_reg = 7'b0010000; // Pattern for 9 (segments a, b, c, d, f, g
on)
            default: seg_reg = 7'b1111111; // Off (all segments off)
        endcase

        // Drive only one digit (rightmost digit is active)
        an_reg = 4'b1110;
    end
endmodule
```

13.7 uart_top.v

```
`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
///
// Company:
// Engineer:
```

```

//
// Create Date: 04/10/2025 12:44:12 PM
// Design Name:
// Module Name: uart
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////
///

module top_uart(
    input clk,           // System clock
    input rst,           // System reset
    input rx_serial,     // Serial input from PC
    output tx_serial     // Serial output to PC
);

    wire [7:0] rx_byte;
    wire rx_done;
    wire tx_start;

    // Instantiate UART receiver
    uart_rx uart_rx_inst(
        .clk(clk),
        .rst(rst),
        .rx_serial(rx_serial),
        .rx_byte(rx_byte),
        .rx_done(rx_done)
    );

    // Instantiate UART transmitter
    uart_tx uart_tx_inst(
        .clk(clk),
        .rst(rst),
        .tx_byte(rx_byte), // Transmit received data
        .tx_start(rx_done), // Start transmission when data is received
        .tx_serial(tx_serial)
    );

```

```

endmodule

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
///
// Company:
// Engineer:
//
// Create Date: 04/10/2025 12:45:11 PM
// Design Name:
// Module Name: uart_rx
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
///

module uart_rx(
    input clk,           // System clock
    input rst,           // System reset
    input rx_serial,      // Serial input from PC
    output reg [7:0] rx_byte, // Received parallel data
    output reg rx_done    // Signal indicating data received
);

    localparam CLKS_PER_BIT = 10416; // 100MHz / 9600 baud
    localparam IDLE = 2'b00;
    localparam START = 2'b01;
    localparam DATA = 2'b10;
    localparam STOP = 2'b11;

    reg [1:0] state;
    reg [15:0] clk_count;
    reg [2:0] bit_index;
    reg [7:0] data_reg;

    always @(posedge clk or posedge rst) begin

```

```

if (rst) begin
    state <= IDLE;
    clk_count <= 0;
    bit_index <= 0;
    rx_done <= 0;
    rx_byte <= 0;
    data_reg <= 8'b0;
end else begin
    case (state)
        IDLE: begin
            rx_done <= 0;
            if (rx_serial == 0) begin // Start bit detected
                state <= START;
                clk_count <= 0;
            end
        end

        START: begin
            if (clk_count == (CLKS_PER_BIT / 2)) begin
                if (rx_serial == 0) begin // Confirm start bit
                    state <= DATA;
                    clk_count <= 0;
                    bit_index <= 0;
                end else begin
                    state <= IDLE;
                end
            end else begin
                clk_count <= clk_count + 1;
            end
        end

        DATA: begin
            if (clk_count == CLKS_PER_BIT) begin
                data_reg[bit_index] <= rx_serial; // Sample data bit
                clk_count <= 0;
                if (bit_index == 7) begin
                    state <= STOP;
                end else begin
                    bit_index <= bit_index + 1;
                end
            end else begin
                clk_count <= clk_count + 1;
            end
        end

        STOP: begin

```

```

        if (clk_count == CLKS_PER_BIT) begin
            rx_byte <= data_reg; // Output received byte
            rx_done <= 1;       // Signal data received
            state <= IDLE;
        end else begin
            clk_count <= clk_count + 1;
        end
    end

    default: state <= IDLE;
endcase
end
end
endmodule

`timescale 1ns / 1ps
module uart_tx(
    input clk,
    input rst,
    input [7:0] tx_byte, // Parallel data to transmit
    input tx_start,      // Signal to start transmission
    output reg tx_serial // Serial output to PC
);

    localparam CLKS_PER_BIT = 10416; // 100MHz / 9600 baud
    localparam IDLE = 2'b00;
    localparam START = 2'b01;
    localparam DATA = 2'b10;
    localparam STOP = 2'b11;

    reg [1:0] state;
    reg [15:0] clk_count;
    reg [2:0] bit_index;
    reg [7:0] data_reg;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            state <= IDLE;
            clk_count <= 0;
            bit_index <= 0;
            tx_serial <= 1;
        end else begin
            case (state)
                IDLE: begin
                    tx_serial <= 1;
                    if (tx_start) begin

```

```

        state <= START;
        data_reg <= tx_byte;
        clk_count <= 0;
    end
end

START: begin
    tx_serial <= 0;
    if (clk_count == CLKS_PER_BIT) begin
        state <= DATA;
        clk_count <= 0;
        bit_index <= 0;
    end else begin
        clk_count <= clk_count + 1;
    end
end

DATA: begin
    tx_serial <= data_reg[bit_index];
    if (clk_count == CLKS_PER_BIT) begin
        clk_count <= 0;
        if (bit_index == 7) begin
            state <= STOP;
        end else begin
            bit_index <= bit_index + 1;
        end
    end else begin
        clk_count <= clk_count + 1;
    end
end

STOP: begin
    tx_serial <= 1;
    if (clk_count == CLKS_PER_BIT) begin
        state <= IDLE;
    end else begin
        clk_count <= clk_count + 1;
    end
end

default: state <= IDLE;
endcase
end
end
endmodule

```

13.8 tb_soc.v

```
`timescale 1ns/1ps
module tb_soc;
    reg clk;
    reg rst;
    reg rx_serial;
    wire tx_serial;
    wire [6:0] seg;
    wire [3:0] an;
    wire status_led;

    // Instantiate the top-level SoC
    soc_top uut (
        .clk(clk),
        .rst(rst),
        .rx_serial(rx_serial),
        .tx_serial(tx_serial),
        .seg(seg),
        .an(an),
        .status_led(status_led)
    );

    // Clock generation: 10 ns period (100 MHz)
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        rx_serial = 1; // idle high for UART
        #100;
        rst = 0;
    end
endmodule
```

13.9 main.c

```
#include <stdint.h>

// Firmware is stored in flash starting at 0x80000000.
#define FLASH_BASE      0x80000000
```

```

// SRAM addresses where the data will be copied.
#define IMAGE_BASE      0x00010000    // MNIST image data
#define WEIGHT1_BASE    0x00020000    // Layer 1 weights
#define WEIGHT2_BASE    0x00030000    // Layer 2 weights
#define WEIGHT3_BASE    0x00040000    // Layer 3 weights

// Base address for memory mapped peripheral registers.
#define MMIO_BASE        0x40000000

// Peripheral register offsets.
#define ACCEL_CTRL_OFFSET 0x00    // Accelerator control register
#define ACCEL_OUT_OFFSET  0x04    // Accelerator output register
#define SEVEN_SEG_OFFSET  0x10    // Seven-seg display register
#define UART_TX_OFFSET    0x20    // UART transmit register

// Define registers by mapping peripheral addresses.
#define ACCEL_CTRL    (*(volatile uint32_t *) (MMIO_BASE + ACCEL_CTRL_OFFSET))
#define ACCEL_OUT     (*(volatile uint32_t *) (MMIO_BASE + ACCEL_OUT_OFFSET))
#define SEVEN_SEG     (*(volatile uint32_t *) (MMIO_BASE + SEVEN_SEG_OFFSET))
#define UART_TX       (*(volatile uint32_t *) (MMIO_BASE + UART_TX_OFFSET))

// Data width note:
// The original flash files used 196 32-bit words for image data, but my
// accelerator expects 784 16-bit words. Writing a 32-bit word produces two 16-bit
// words,
// so we double the number of 32-bit words to copy.
// The same doubling applies to the weight arrays.
#define IMAGE_WORDS    392    // 392 x 32-bit words = 784 halfwords
#define WEIGHT1_WORDS  25120  // (12560 * 2)
#define WEIGHT2_WORDS  1040   // (520 * 2)
#define WEIGHT3_WORDS  166    // (83 * 2)

// The firmware code is linked at FLASH_BASE.
// _etext is defined by the linker as the end of the firmware code
extern char _etext[];
#define FIRMWARE_SIZE    ((uint32_t)_etext - FLASH_BASE)

// Function to copy a block of memory.
void copy_memory(uint32_t src_addr, uint32_t dest_addr, uint32_t num_words) {
    volatile uint32_t *src = (volatile uint32_t *)src_addr;
    volatile uint32_t *dest = (volatile uint32_t *)dest_addr;
    for (uint32_t i = 0; i < num_words; i++) {
        dest[i] = src[i];
    }
}

```

```

int main(void) {
    // Copy image data (flash is laid out with firmware first, then image data)
    copy_memory(FLASH_BASE + FIRMWARE_SIZE, IMAGE_BASE, IMAGE_WORDS);

    // Copy weights data to SRAM.
    copy_memory(FLASH_BASE + FIRMWARE_SIZE + IMAGE_WORDS * 4, WEIGHT1_BASE,
WEIGHT1_WORDS);
    copy_memory(FLASH_BASE + FIRMWARE_SIZE + (IMAGE_WORDS + WEIGHT1_WORDS) * 4,
WEIGHT2_BASE, WEIGHT2_WORDS);
    copy_memory(FLASH_BASE + FIRMWARE_SIZE + (IMAGE_WORDS + WEIGHT1_WORDS +
WEIGHT2_WORDS) * 4, WEIGHT3_BASE, WEIGHT3_WORDS);

    // Start the accelerator by writing '1' to its control register.
    ACCEL_CTRL = 1;

    // Wait until the accelerator clears the control register (indicating nn finished
inferencing)
    while (ACCEL_CTRL != 0) {}

    // Retrieve the prediction result and output it
    uint32_t prediction = ACCEL_OUT;
    SEVEN_SEG = prediction; // Display on seven seg
    UART_TX = prediction;   // Send over UART

    // Stay here forever
    while (1) {}

    return 0;
}

```

13.10 start.S

```

.section .init
.globl _start
_start:
    la      sp, _stack_top    # Load stack pointer from _stack_top defined in the
linker script
    call    main              # Call main()
1: j       1b                # Infinite loop if main returns

```

13.11 linker.ld

```

OUTPUT_ARCH( "riscv" )

```

```

ENTRY(_start)

MEMORY
{
    FLASH (rx) : ORIGIN = 0x80000000, LENGTH = 256K /* Flash for firmware */
    SRAM (rwx) : ORIGIN = 0x00010000, LENGTH = 128K /* SRAM for data, image,
weights, stack */
}

SECTIONS
{
    /* Firmware code and read-only data in flash */
    .text : {
        _start = .;
        *(.init) /* Startup code from start.S */
        *(.text*) /* All code */
        *(.rodata*) /* Read-only data */
        _etext = .; /* End of firmware code */
    } > FLASH

    /* Initialized data in SRAM; its load image in flash comes after _etext */
    .data : AT(_etext) {
        _data = .;
        *(.data*)
        _edata = .;
    } > SRAM

    /* Uninitialized data in SRAM */
    .bss : {
        _bss = .;
        *(.bss*)
        *(COMMON)
        _ebss = .;
    } > SRAM

    /* Reserve stack space: define the bottom and top of the stack based on _ebss
*/
    _stack_bottom = ALIGN(8, _ebss);
    _stack_top = _stack_bottom + 0x1000; /* 4 KB of stack */
}

```

13.12 re-assembled assembly.txt

```
firmware.elf:      file format elf32-littleriscv
```

Disassembly of section .text:

```

80000000 <_start>:
80000000:    80011117        auipc   sp,0x80011
80000004:    00010113        mv      sp,sp
80000008:    00000097        auipc   ra,0x0
8000000c:    034080e7        jalr    52(ra) # 8000003c <main>
80000010:    0000006f        j       80000010 <_start+0x10>

80000014 <copy_memory>:
80000014:    00261613        slli    a2,a2,0x2
80000018:    00000793        li      a5,0
8000001c:    00c79463        bne     a5,a2,80000024 <copy_memory+0x10>
80000020:    00008067        ret
80000024:    00f506b3        add     a3,a0,a5
80000028:    0006a683        lw      a3,0(a3)
8000002c:    00f58733        add     a4,a1,a5
80000030:    00478793        addi    a5,a5,4
80000034:    00d72023        sw      a3,0(a4)
80000038:    fe5ff06f        j       8000001c <copy_memory+0x8>

8000003c <main>:
8000003c:    ff010113        addi    sp,sp,-16 # 10ff0 <_bss+0xff0>
80000040:    80000537        lui     a0,0x80000
80000044:    00812423        sw      s0,8(sp)
80000048:    18800613        li      a2,392
8000004c:    0e050413        addi    s0,a0,224 # 800000e0 <_etext>
80000050:    000105b7        lui     a1,0x10
80000054:    0e050513        addi    a0,a0,224
80000058:    00112623        sw      ra,12(sp)
8000005c:    00000097        auipc   ra,0x0
80000060:    fb8080e7        jalr    -72(ra) # 80000014 <copy_memory>
80000064:    00006637        lui     a2,0x6
80000068:    22060613        addi    a2,a2,544 # 6220 <_bss-0x9de0>
8000006c:    000205b7        lui     a1,0x20
80000070:    62040513        addi    a0,s0,1568
80000074:    00000097        auipc   ra,0x0
80000078:    fa0080e7        jalr    -96(ra) # 80000014 <copy_memory>
8000007c:    00019537        lui     a0,0x19
80000080:    ea050513        addi    a0,a0,-352 # 18ea0 <_stack_top+0x7ea0>
80000084:    41000613        li      a2,1040
80000088:    000305b7        lui     a1,0x30
8000008c:    00a40533        add     a0,s0,a0
80000090:    00000097        auipc   ra,0x0
80000094:    f84080e7        jalr    -124(ra) # 80000014 <copy_memory>

```

```

80000098:    0001a537          lui    a0,0x1a
8000009c:    ee050513          addi   a0,a0,-288 # 19ee0 <_stack_top+0x8ee0>
800000a0:    0a600613          li     a2,166
800000a4:    000405b7          lui    a1,0x40
800000a8:    00a40533          add    a0,s0,a0
800000ac:    00000097          auipc  ra,0x0
800000b0:    f68080e7          jalr   -152(ra) # 80000014 <copy_memory>
800000b4:    400007b7          lui    a5,0x40000
800000b8:    00100713          li     a4,1
800000bc:    00e7a023          sw     a4,0(a5) # 40000000
<_stack_top+0x3ffef000>
800000c0:    0007a703          lw     a4,0(a5)
800000c4:    fe071ee3          bnez   a4,800000c0 <main+0x84>
800000c8:    400007b7          lui    a5,0x40000
800000cc:    0047a783          lw     a5,4(a5) # 40000004
<_stack_top+0x3ffef004>
800000d0:    40000737          lui    a4,0x40000
800000d4:    00f72823          sw     a5,16(a4) # 40000010
<_stack_top+0x3ffef010>
800000d8:    02f72023          sw     a5,32(a4)
800000dc:    0000006f          j      800000dc <main+0xa0>

```

13.13 weights_and_image.py

```

import os
import numpy as np
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Input
from PIL import Image

print("Loading MNIST dataset...")
(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

print("Creating and training model...")
model = Sequential(
    [
        Input(shape=(28, 28)),
        Flatten(),
        Dense(64, activation="relu"),
        Dense(32, activation="relu"),
        Dense(10, activation="linear"),
    ]
)

```

```

model.compile(
    optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"],
)
model.fit(x_train, y_train, epochs=5, validation_split=0.1, verbose=1)
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=1)
print("Test accuracy:", test_acc)

def to_fixed_hex(x):
    fixed = int(round(x * 256))
    fixed &= 0xFFFF
    return format(fixed, "04x")

img_arr = x_test[0] # Image of a handdrawn 7

# Export the test image (flattened to 784 pixels)
test_img_flat = img_arr.flatten()
with open("image.hex", "w") as f:
    for px in test_img_flat:
        f.write(to_fixed_hex(px) + "\n")
print(f"Exported {len(test_img_flat)} pixels to image.hex")

def process_layer(layer):
    w, b = (
        layer.get_weights()
    ) # w shape: (in_features, out_features) b shape: (out_features,)
    w = w.T # each row corresponds to one neuron
    rows = []
    for i in range(w.shape[0]):
        row = [b[i]] + list(w[i])
        rows.append(row)
    return np.array(rows)

# Get weights from Dense layers in order.
all_weights = []
for layer in model.layers:
    if isinstance(layer, Dense):
        proc = process_layer(layer)
        all_weights.append(proc)

fixed_weights = []
for layer in all_weights:

```

```

    for row in layer:
        for val in row:
            fixed_weights.append(to_fixed_hex(val))

# Total expected count: 64*785 + 32*65 + 10*33 = 52650.
with open("weights.hex", "w") as f:
    for hex_val in fixed_weights:
        f.write(hex_val + "\n")
print(f"Exported {len(fixed_weights)} weights to weights.hex")

# Test prediction on the custom image.
pred = model.predict(np.expand_dims(img_arr, axis=0))
print("Model prediction (class):", np.argmax(pred, axis=1)[0])
for i, logit in enumerate(pred[0]):
    print(f"Output[{i}] = {logit:.4f}, fixed: {to_fixed_hex(logit)}")

```